Real-Time Object Detection and Distance Mapping

A PROJECT REPORT

Submitted by

KINTHALI SAKETH[RA2111003011009]
PUDI SAI KRISHNA[RA2111003011033]

Under the Guidance of

DR. RAGHAVENDRA V

(Assistant Professor, Department of Computing Technologies)

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING

DEPARTMENT OF COMPUTING TECHNOLOGIES
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE ANDTECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2025

Department of Computing Technologies
SRM Institute of Science & Technology
Own Work* Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree/ Course : **Bachelor of Technology in Computer Science and Engineering**

Student Name : **Kinthali Saketh, Pudi Sai Krishna**

Registration Number : **RA2111003011009, RA2111003011033**

Title of Work : **Real-Time Object Detection and Distance**

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism**, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website.

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.

KINTHALI SAKETH :

PUDI SAI KRISHNA :

DATE :

If you are working in a group, please write your registration numbers and sign with the date for every student in your group.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 18CSP109L - Major Project report titled "**REAL-TIME OBJECT DETECTION AND DISTANCE MAPPING**" is the bonafide work of "**KINTHALI SAKETH [RA2111003011009], PUDI SAI KRISHNA [RA2111003011033],"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. RAGHAVENDRA V

SUPERVISOR
ASSISTANT PROFESSOR
DEPARTMENT OF
COMPUTING TECHNOLOGIES

SIGNATURE

Dr. G. NIRANJANA

PROFESSOR &HEAD
DEPARTMENT OF
COMPUTING TECHNOLOGIES

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. T. V. Gopal,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson, School of Computing and **Dr. C. Lakshmi,** Professor and Associate Chairperson, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. G. Niranjana**, Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, **Dr. Arthi B, Dr. Saravanan P, Dr Raghavendra V,** Department of Computing Technologies, SRM Institute of Science and Technology, for theirinputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. Iniyan S**, Department of Computing Technologies, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. Raghavendra V**, Department of Computing Technologies, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff and students of Computing Technologies, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

KINTHALI SAKETH(RA2111003011009)
PUDI SAI KRISHNA (RA2111003011033)

ABSTRACT

The "Real-Time Object Detection and Distance Mapping" project is an intelligent vision system that can detect, identify, and calculate the relative distance of objects in the context of real-world scenarios. The project makes use of the YOLOv8 deep learning framework, which boasts high accuracy and is capable of delivering real-time performance, to identify differing objects within stationary images and ongoing video streams. Using calibrated focal lengths and object size estimations, the system calculates the physical distance of the detected objects from the camera to a high degree of accuracy.

The system consists of several modules: video-based multi-object tracking with distance annotations, static image object detection, and webcam monitoring in real time with integrated audio feedback. Bounding box detection, centre point detection, pixel-to-meter translation, and visual feedback overlays are used for processing video input in real time, while an optionally integrated text-to-speech interface delivers auditory warnings for users based on their proximity to safety thresholds. The multi-feedback mode offers increased situational awareness and availability for users, including potential benefits for visually impaired individuals.

Moreover, the system utilizes performance improvement techniques like dynamic resizing of frames, multithreaded speech processing, and tracking of frames per seconds to maintain smooth performance regardless of the hardware configuration. Through the use of deep learning, computer vision, and mapping from distances, the project exhibits an effective and scalable application for use in autonomous navigation, traffic monitoring, industrial safety, human-machine interaction, and assistive technologies.

On the whole, this work provides an effective framework to bridge the gap between object recognition and spatial awareness and extends what intelligent vision systems that operate in real time are capable of accomplishing.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

YOLO	You Only Look Once
YOLOv8	You Only Look Once version 8
FPS	Frames Per Second
SDG	Sustainable Development Goal
AI:	Artificial Intelligence
LiDAR	Light Detection and Ranging
SLAM	Simultaneous Localization and Mapping
RGB	Red Green Blue (color model)
MS Planner	Microsoft Planner
COCO	Common Objects in Context (dataset)
IDE	Integrated Development Environment
Venv	Virtual Environment (Python)
NMS	Non-Maximum Suppression
MAP	Mean Average Precision

CHAPTER 1

INTRODUCTION

1.1 Introduction

Computer vision has seen dramatic improvement with advances from deep learning-based object detection models. Among these, YOLO (You Only Look Once)-based models have emerged as the de facto choice for object detection in real time, offering an optimum trade-off between speed and precision. While object detection provides a system with the ability to detect and localize an object within a scene, it lacks the ability to perceive spatial relationships among the detected object and the camera. This shortcoming is overcome by distance mapping, providing invaluable spatial insight that helps improve decisions across applications from autonomous systems to assistive technologies for humans.

The "Real-Time Object Detection and Distance Mapping" project is one that brings these two fundamental abilities together to form an integrated, real-time system. The project utilizes YOLOv8, also one of the newer versions of the YOLO system, for object detection, which is characterized by its lightweight design, high-speed inference, and high accuracy, ideal for use for real-world applications on commodity hardware without the need for costly GPUs.

The system makes use of input data from various sources, which can contain pre-recorded video, live streams from a webcam, and static images. When an object is detected, the system computes an estimate of the distance from the object to the camera using geometric calculations from known physical object measurements, pixel measurements within the acquired image, and parameters of the camera such as focal length. The methods used to estimate the distances include pixel-to-meter scaling and depth estimation models based on focal lengths. Additionally, to enhance the user-friendliness and interactivity of the system, the project adds an audio feedback system driven by a text-to-speech engine. The system delivers auditory alerts regarding obstacles within reach, thereby expanding the application areas of the system to safety-critical domains or visually impaired people.

The major technical elements of the system are:

- YOLOv8 object detection fast and accurate model.
- Realtime video processing including bounding box extraction, centroid tracking and resizing of frames for performance optimization.
- Mathematically calculating distances using the methods of focal length estimation and pixel scaling.
- Speech synthesis for real-time auditory alerts.
- Performance monitoring using calculation of FPS (Frames Per Second) and window scaling.

It is this combination of object detection and distance estimation that makes it possible to have a system that not only senses its surroundings but also interprets the spatial dynamics of its environment. These are essential abilities for future advanced autonomous devices, collision avoidance systems, and intelligent monitoring systems.

1.2 Motivation

The motivation behind the "Real-Time Object Detection and Distance Mapping" project is to address a key limitation in current object detection systems: the inability to understand how far away detected objects are from the camera. While object detection models like YOLO are great at quickly identifying objects in real time, they don't provide any sense of how far those objects are from the camera. This lack of spatial awareness can be a major problem for applications like self-driving cars, safety systems, and assistive technologies.

Adding Spatial Awareness to Object Detection:

- While object detection systems can tell you what an object is, they don't tell you how far away it is. This project adds that missing piece by using distance mapping, allowing the system to not only detect objects but also understand their proximity. This extra information can be used to make better, safer decisions in real-world situations.

Real-World Applications:

- The goal of the project is to make object detection more useful in the real world. Whether it's for self-driving cars, helping people with visual impairments, or improving safety in factories, knowing how far away an object is makes all the difference. By combining object recognition with distance estimation, the system can be used in more practical and life-saving ways.

Efficiency and Accessibility:

- YOLOv8, the object detection model used in this project, is designed to run quickly and efficiently, even on regular, off-the-shelf hardware. This means the system doesn't require expensive equipment like high-end GPUs, making it more affordable and accessible to a wider range of users.

Making It Easier to Use:

- To make the system even more user-friendly, the project includes a text-to-speech feature that gives auditory alerts about objects' proximity. This is especially helpful for visually impaired individuals, allowing them to interact with the system through sound rather than relying solely on visual cues.

Optimizing for Real-Time Performance:

- The system is designed to process video streams in real time without lag, making sure that the system can handle live situations without delays. By using techniques like resizing frames and processing video in parallel, the system can deliver smooth, reliable performance even on less powerful hardware.

Wide Range of Uses:

- By combining object detection with distance mapping, the system can be used in a variety of fields, from self-driving cars to smart safety systems. This flexibility makes it a useful tool for many different industries and applications, helping improve everything from personal safety to automation.

1.3 Sustainable Development Goal

The "Real-Time Object Detection and Distance Mapping" system sits at the intersection of advanced technology and real-world impact, supporting several **Sustainable Development Goals (SDGs)**. By creating more inclusive, accessible, and safer environments, your project helps contribute to sustainability in both technological and societal terms. It has the potential to change the way we think about autonomous systems, safety, and assistive technologies, making these innovations more accessible, efficient, and meaningful for communities around the world.

Here's how your project aligns with some key SDGs:

SDG 3: Good Health and Well-being

- **How your project fits**: Your system has the potential to improve life for people with visual impairments, thanks to its auditory feedback system. This helps users navigate more safely, reducing accidents and enhancing their independence. It's particularly valuable in health-related fields, such as helping individuals in public spaces or providing safety in medical environments.

- **Impact**: By making technology more accessible and safer, your system supports a healthier, more inclusive society.

SDG 9: Industry, Innovation, and Infrastructure

- **How your project fits**: The use of advanced deep learning (YOLOv8) for real-time object detection and distance mapping can be applied in various industries like autonomous navigation, industrial safety, and transportation. By improving real-time video processing and distance estimation, your system enhances the efficiency and safety of operations in sectors where human-machine interaction is crucial.

- **Impact**: Your system promotes innovation and helps build safer, smarter infrastructure, driving progress in fields like manufacturing, robotics, and transportation.

SDG 11: Sustainable Cities and Communities

- **How your project fits**: The system's use in autonomous vehicles, traffic monitoring, and city safety can greatly improve urban life. It can help prevent accidents, optimize traffic flow, and improve city planning. Additionally, assistive features make public spaces more accessible for everyone, including those with disabilities.

- **Impact**: By integrating this system into urban infrastructure, your project can help create safer, more efficient, and inclusive cities.

SDG 17: Partnerships for the Goals

- **How your project fits**: Collaborating with organizations, institutions, and businesses focused on assistive tech, smart cities, or autonomous vehicles can help your project reach more people. Partnerships can also enhance the system's impact, scaling it to benefit a broader audience across different sectors.

- **Impact**: Your project fosters collaboration and innovation, helping to spread the benefits of this technology and contribute to the achievement of the SDGs.

1.4 Problem Statement

In today's world, many technologies rely on being able to detect and understand objects around them. While models like YOLO have made huge strides in quickly identifying objects in real time, they still face a big limitation: they can't tell us how far away these objects are from the camera or sensor. This lack of spatial awareness creates problems in areas where knowing the distance to an object is crucial—like in autonomous driving, safety systems, and assistive technologies.

- **Self-driving cars** need to not only detect pedestrians or other vehicles but also understand how far away they are to avoid crashes.
- **Assistive technology** for visually impaired individuals could be much more helpful if it could detect objects in their path and tell them how close those objects are, helping them move around more safely.
- **Industrial systems** that track equipment and people need to know how close objects are to each other to avoid accidents and optimize operations.

Right now, most systems that can detect objects lack the ability to calculate the distance to those objects in real time. This missing piece limits their usefulness, especially in situations where split-second decisions are needed, such as in autonomous navigation or providing safety alerts.

1.5 Objective of the Project

The goal of the "Real-Time Object Detection and Distance Mapping" project is to create a system that not only detects objects quickly and accurately but also estimates how far those objects are from the camera or sensor, all in real time. By using the YOLOv8 deep learning model, the system can identify various objects—like people, vehicles, or obstacles—and then calculate their distance using methods like pixel-to-meter scaling and focal length estimation. This will give a clearer understanding of the environment, making it easier to interact with and respond to what's around you.

The system will provide both visual and auditory feedback. It will display the location of detected objects through bounding boxes and also use text-to-speech technology to warn users when objects are getting too close. This is particularly useful in scenarios like autonomous driving, helping visually impaired individuals navigate, or improving safety in industrial settings—any situation where knowing the proximity of objects can help make better, safer decisions.

To make sure the system works well on a variety of devices, it will be optimized for performance. Techniques like resizing video frames dynamically and using multithreaded processing will help ensure smooth operation, even on less powerful hardware. The ultimate aim of this project is to combine object detection with spatial awareness, making technology safer, more efficient, and more accessible in real-world applications.

1.6 Project Domain

The "Real-Time Object Detection and Distance Mapping" system blends computer vision, deep learning, and spatial awareness to create a powerful tool for understanding the world around us. With computer vision, the system can detect and recognize objects in images or videos, while deep learning models like YOLOv8 allow it to do so accurately and in real time. This is especially important in situations like autonomous driving, robotics, or surveillance, where quickly identifying objects is essential. The system's ability to process visual data instantly opens up a wide range of practical uses, from self-driving cars to security systems.

A major part of the project is its ability to understand spatial awareness and calculate distances. Once an object is detected, the system can estimate how far it is from the camera using techniques like pixel-to-meter scaling and depth estimation. This extra layer of information adds context to the visual data, allowing the system to understand not just what's around it, but how far away things are. This capability is crucial for applications like autonomous navigation, where knowing the exact distance to an object is key for making safe decisions, or assistive technologies for those with visual impairments, helping them navigate with more confidence.

Additionally, this project makes significant strides in improving assistive technologies and safety in industrial settings. For visually impaired individuals, the real-time object detection and distance mapping can provide valuable audio alerts to help them navigate their environment more safely. In industrial environments, the technology can be used to monitor the proximity of workers to equipment or obstacles, reducing the risk of accidents and enhancing safety. By combining object detection with spatial awareness, this project is making systems smarter, safer, and more efficient across a variety of industries and applications.

1.7 Scope of the Project

The scope of the "Real-Time Object Detection and Distance Mapping" project focuses on creating a system that can detect objects in real time and estimate their distance from the camera. By using the YOLOv8 deep learning model, the system will identify objects like people, vehicles, or obstacles and calculate their distance through methods like pixel-to-meter scaling. This is particularly useful for applications like autonomous driving, robotics, and assistive technologies. The system will also provide auditory feedback for visually impaired individuals, alerting them to nearby objects and helping them navigate more safely. This aspect of the project aims to enhance accessibility and independence for those with disabilities.

In addition to assistive technologies, the system will have practical applications in industrial safety, where it can monitor the proximity of workers to machinery or hazardous zones, helping to prevent accidents. The project will ensure real-time video processing and performance optimization, allowing the system to work efficiently even on devices with limited hardware. With a focus on scalability, the system will be adaptable for use in various industries, from autonomous navigation to smart city infrastructure, making it a versatile tool that can be integrated into different environments and contribute to improving safety and efficiency across multiple sectors.

1.8 Methodology

Data Acquisition

The project utilizes pre-trained YOLOv8 models (yolov8m.pt, yolov8n.pt) for the purpose of object detection. These models have been developed using extensive datasets to identify a wide range of objects. The dataset designated for fine-tuning or validation is detailed in the coco.txt file, which presumably includes class labels and relevant metadata. Input data for testing and validation comprises image and video files, such as bus.jpg, 33.mp4, and 34.mp4. By incorporating both image and video formats, the model's performance is assessed in both static and dynamic environments, thereby evaluating its robustness and accuracy across various scenarios.

Preprocessing

Prior to executing object detection, the input data is subjected to preprocessing to enhance its suitability for model inference. Images and video frames are imported into the system and resized to conform to the input dimensions specified by YOLOv8. This step guarantees that the model can process the data efficiently while preserving essential details. Additional modifications, including normalization and color space conversion, may be implemented to ensure a consistent input format. For real-time detection, video streams are analyzed frame by frame utilizing OpenCV. This enables the model to evaluate live footage and identify objects in real-time, which is vital for applications like surveillance and autonomous navigation. The preprocessing phase is critical in ensuring that the input data is organized and primed for precise object detection.

Model Implementation

The object detection pipeline is developed utilizing the YOLOv8 deep learning framework. Various Python scripts, including object_detection.py, real_time.py, and video_with_distance.py, cater to distinct applications. The object_detection.py script focuses on identifying objects within static images, while real_time.py facilitates live detection through a connected camera. Additionally, the video_with_distance.py script enhances functionality by integrating distance estimation, which is beneficial for scenarios requiring measurement of object proximity. The YOLOv8 model processes the input data by segmenting it into a grid and predicting bounding boxes, class labels, and confidence scores for the identified objects. By harnessing the real-time processing capabilities of YOLOv8, the system guarantees swift and precise object recognition across various contexts.

Post-processing

Following the object detection phase, the results are subjected to post-processing to improve their clarity and usability. Detected objects are emphasized by applying bounding boxes and labels to the original images or video frames. This visualization aids users in comprehending the model's predictions and evaluating its accuracy. The refined outputs are subsequently stored in various formats, including images (output_detected.jpg) and videos, for documentation and further examination. The post-processing stage also involves filtering detections according to confidence thresholds to reduce false positives and enhance precision. Moreover, additional features, such as tracking objects across frames, may be integrated to further refine detection outcomes and offer deeper insights in dynamic environments.

Evaluation and Optimization

To guarantee the model's effectiveness, its performance is assessed through various configurations and optimization methods. The analysis focuses on accuracy and detection speed by testing different confidence thresholds and non-maximum suppression (NMS) parameters. The choice between yolov8m.pt and yolov8n.pt hinges on the balance between model size and accuracy; yolov8m.pt provides greater precision, whereas yolov8n.pt is designed for enhanced speed. Performance indicators such as precision, recall, and mean average precision (MAP) are employed to evaluate detection quality. Optimization approaches may involve fine-tuning the model with a custom dataset, minimizing computational demands, and modifying hyperparameters to boost efficiency. These measures contribute to improving the model's accuracy and its adaptability in various environments.

Deployment

The concluding phase of the project focuses on the implementation of the object detection system for real-world applications. This system can be integrated into an application that facilitates real-time detection via live camera feeds, making it ideal for uses in security surveillance, traffic monitoring, and industrial automation. Future enhancements may include the integration of the model into web or mobile applications, enabling users to conduct object detection from remote locations. The deployment phase also takes into account hardware limitations, ensuring that the system is compatible with edge devices like Raspberry Pi or NVIDIA Jetson for real-time processing. By providing access to the system across multiple platforms, the project enhances its functionality and influence in various fields.

1.9 MS PLANNER

Microsoft Planner is a task management tool within the Microsoft 365 suite, designed to help teams collaborate, organize, and track work visually. It provides a simple, intuitive interface where users can create "Plans" with multiple tasks, assign them to team members, set due dates, and add relevant details such as checklists, attachments, and comments. Tasks are organized in a board format with customizable columns, enabling users to visually manage progress and see which tasks are completed or still pending. Planner integrates seamlessly with other Microsoft tools like Teams and Outlook, offering notifications, calendar syncing, and an overview of tasks within the broader Microsoft ecosystem, making it easy for teams to stay coordinated and efficient.

Initiating: This phase focuses on setting the foundation for the project. Two tasks have been completed here: "Identify goals and objectives," which is marked as awaiting review and ensures that the project goals are clearly defined and aligned with stakeholder expectations. Another completed task, "Develop strategies and plans," involves creating a high-level strategy to guide the project through its lifecycle, making sure all steps are systematically planned.

Planning: In this phase, the main focus is on detailed project planning and preparation. The only task completed here is "Data set Evaluation," also marked as awaiting review. This task involves assessing the quality, relevance, and completeness of the data to be used, ensuring that it meets the project's requirements. This evaluation is crucial for avoiding potential data issues during later stages of the project.

Executing: This phase is where the actual work on the project deliverables occurs. Several tasks have been completed, including "Object Detection," "Model Testing," "Model Training," and "Data Handling." Each of these tasks represents a key step in the execution of the project, from handling the data to building and testing predictive models. These activities directly contribute to the project's objectives by ensuring that the models are trained, tested, and prepared for predictions as per project requirements.

Monitoring and Controlling: This phase involves keeping the project on track and ensuring that progress aligns with the planned objectives. Repeated tasks, "Report Weekly Status," indicate a consistent effort to provide regular project updates. These reports are crucial for transparency and accountability, keeping stakeholders informed on the project's progression. Another completed task, "Conduct team performance review," ensures the team's work aligns with the project's quality standards. showcasing his role in maintaining project visibility and control.

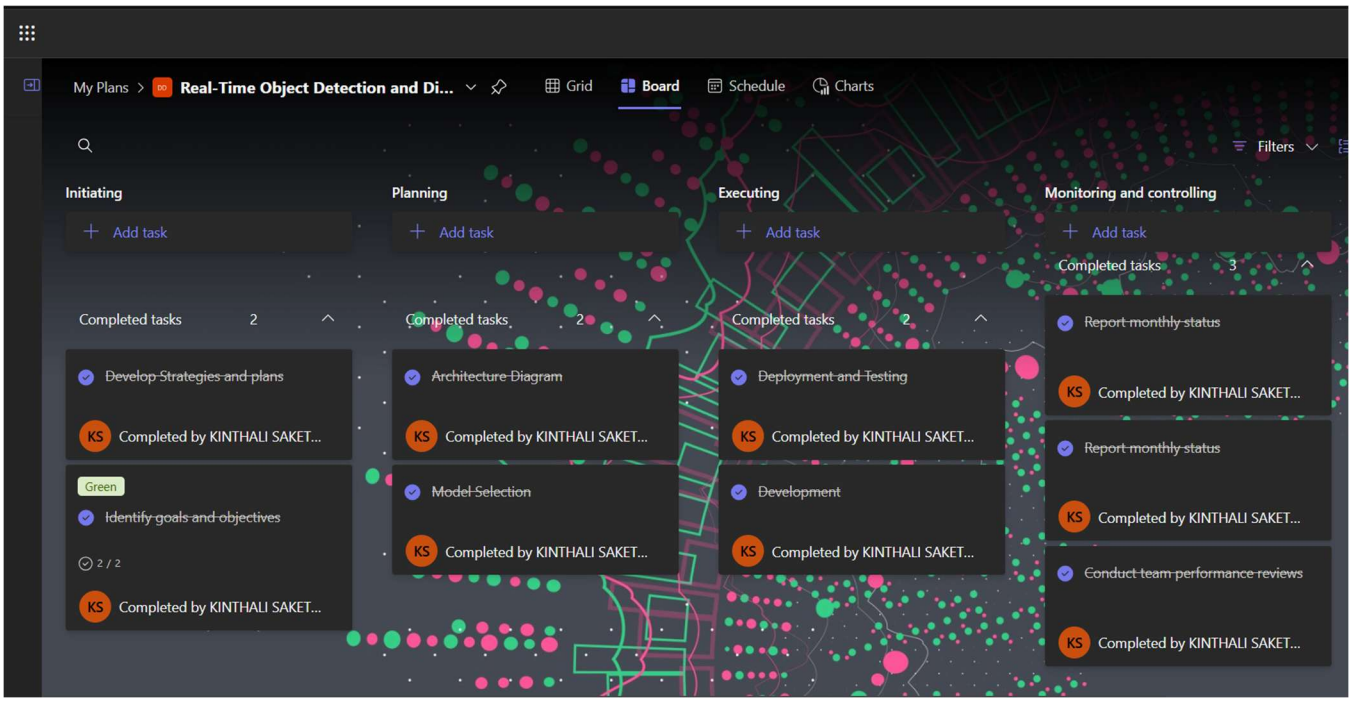
Check for Plagiarism: This task ensures that the project's deliverables are original and free from any form of plagiarism. It's a critical step in maintaining the integrity and credibility of the work, particularly if the project involves research or publication.

Evaluation Report: This task involves compiling a report that evaluates the project's overall performance, covering key aspects such as goals achieved, challenges faced, and lessons learned. This report serves as a formal assessment and provides valuable insights for future projects.

Prepare Final Conference Paper: For projects that are research-oriented or require dissemination of findings, preparing a final conference paper is essential. This task involves drafting a polished, comprehensive paper that summarizes the project's objectives, methodology, and results, ready for presentation or publication.

Evaluating Performance: This task focuses on assessing the performance of the team, processes, and outcomes. By evaluating performance, the project team can identify strengths, weaknesses, and areas for improvement, contributing to personal and organizational growth.

Project Results: This task documents the final outcomes and results of the project. It provides a summary of what was achieved, detailing how well the project met its original goals and objectives. This is a crucial record for stakeholders and serves as a reference for future projects.



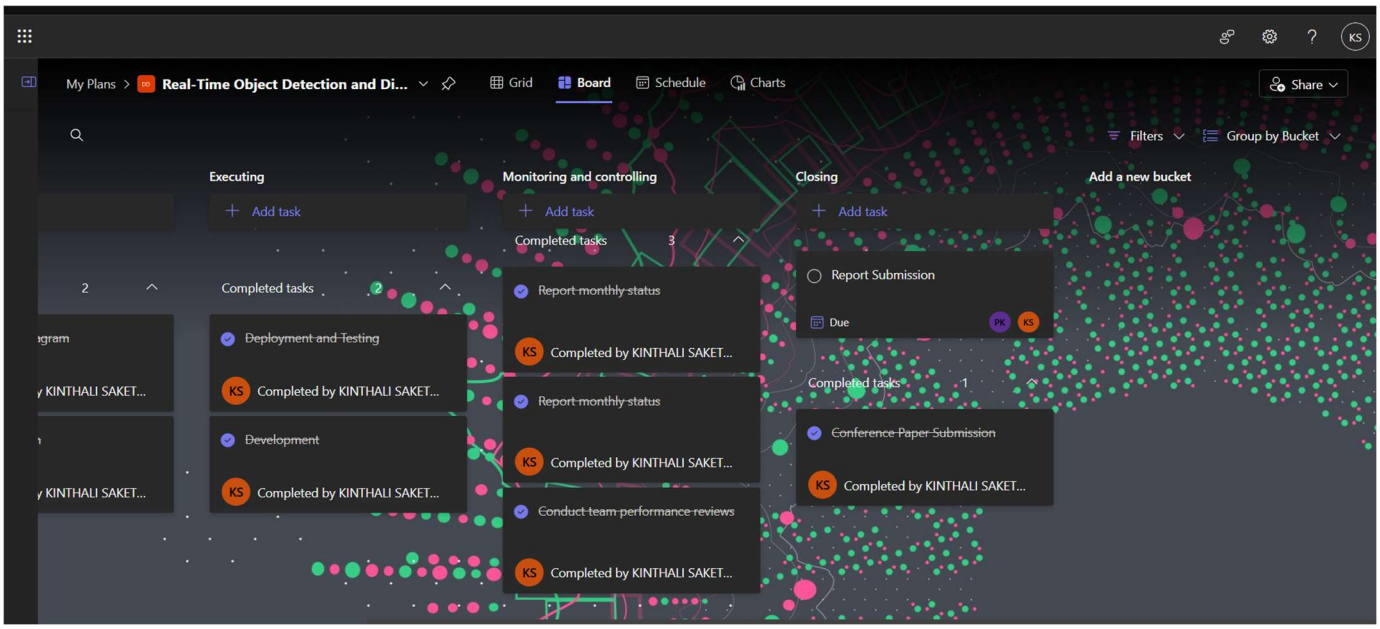


Figure 1.9.1. MS PLANNER

CHAPTER 2

LITERATURE SURVEY

Appiah and Mensah (2024) explored object detection in adverse weather conditions for autonomous vehicles, highlighting the challenges posed by rain, fog, and snow. Their study demonstrated that deep learning models, combined with sensor fusion techniques, can significantly improve object recognition accuracy under harsh environmental conditions.

Yeong et al. (2021) provided a comprehensive review of sensor and sensor fusion technologies used in autonomous vehicles. Their study underscored the importance of integrating multiple sensor types, such as LiDAR, cameras, and radar, to enhance vehicle perception. Despite advancements, they noted that real-time data processing and cost constraints remain significant hurdles in large-scale implementation.

Wason et al. (2024) investigated the convergence of artificial intelligence and mechatronics in autonomous driving. They emphasized the role of AI in decision-making, path planning, and obstacle avoidance, demonstrating that integrating mechatronic systems with AI can enhance vehicle efficiency.

Ghintab and Hassan (2023) focused on localization for self-driving vehicles using deep learning networks and RGB cameras. Their findings revealed that deep learning-based localization outperforms traditional GPS-based methods in urban environments. Nevertheless, their research highlighted that model accuracy is dependent on high-quality training datasets and real-time computational efficiency.

Hasanujjaman et al. (2023) examined sensor fusion in autonomous vehicles, particularly in traffic surveillance systems. They proposed an AI-based approach for detection and localization using traffic cameras, improving vehicle tracking accuracy. While their method demonstrated promising results, scalability and integration with existing infrastructure were noted as potential limitations.

Hacohen et al. (2022) conducted a survey on technological gaps in autonomous driving using trend analysis from Google Scholar and Web of Science. They identified key challenges, including regulatory issues, ethical concerns, and robustness of AI models.

Mahima et al. (2024) reviewed adversarial attacks and countermeasures in 3D perception for autonomous vehicles. Their research showed that autonomous systems are vulnerable to adversarial perturbations, which can mislead object detection models. They suggested robust defense mechanisms, including adversarial training and sensor fusion, to enhance security.

Huang et al. (2025) introduced OpenGV 2.0, a motion prior-assisted calibration and SLAM framework for vehicle-mounted surround-view systems. Their method improved localization accuracy by leveraging motion priors, reducing errors in real-world navigation scenarios. However, they acknowledged that real-time implementation requires substantial computational power.

Yan et al. (2022) developed Agenti2p, an optimization framework for image-to-point cloud registration using behavior cloning and reinforcement learning. Their approach enhanced the accuracy of sensor alignment in autonomous vehicles. While their results were promising, they pointed out that real-world testing in dynamic environments is needed to validate the model's robustness.

CHAPTER 3

PROJECT DESCRIPTION

3.1 Existing System

Currently, many existing systems focus on either object detection or distance estimation, but few combine both features in a seamless, real-time manner. For object detection, popular deep learning models like YOLO, SSD, and Faster R-CNN are commonly used. These models are highly effective at detecting objects in images and videos, with YOLO being particularly favored for its speed and efficiency. It's widely used in applications like autonomous vehicles and surveillance systems. However, while these systems are great at identifying objects, they don't typically offer any information about how far away those objects are from the camera.

When it comes to estimating distances, existing solutions usually rely on specialized hardware such as LIDAR, stereo vision cameras, or structured light systems. While these technologies can provide accurate distance measurements, they tend to be expensive, complex, and often require high-powered hardware to function in real-time. Some systems use depth estimation models or stereo cameras to calculate distances, but these methods can be computationally expensive and may not always work reliably in fast-moving or dynamic environments.

There are also some hybrid systems that attempt to combine object detection and distance estimation, but they often require expensive, heavy-duty hardware or complex setups. For instance, autonomous vehicles may use a combination of cameras, radar, and LIDAR to detect objects and estimate distances, but these systems are typically bulky and costly. This is where your "Real-Time Object Detection and Distance Mapping" project stands out. By combining deep learning-based object detection with simple yet effective techniques for distance estimation, like pixel-to-meter scaling and focal length estimation, your project aims to provide a more accessible, efficient, and scalable solution. It eliminates the need for specialized hardware, making real-time object detection and distance mapping possible on more affordable and lightweight systems.

3.2 Proposed System

The proposed system for the "Real-Time Object Detection and Distance Mapping" project aims to bridge the gap between object detection and spatial awareness in a way that is both efficient and accessible. Using the YOLOv8 deep learning model, the system will be able to detect objects in real time, such as people, vehicles, or obstacles, within images or video streams. This real-time detection will be combined with a unique feature: distance estimation. By using geometric methods like pixel-to-meter scaling and focal length estimation, the system will calculate the physical distance between the detected objects and the camera, providing valuable spatial awareness.

In addition to identifying objects and calculating their distances, the system will offer both visual and auditory feedback. Visual feedback will be provided through bounding boxes around detected objects, allowing users to see their locations clearly. For added accessibility, particularly for visually impaired users, the system will also incorporate a text-to-speech feature that gives real-time audio alerts about the proximity of objects. This makes it especially useful in assistive technologies, autonomous navigation, and industrial safety, where being aware of nearby objects in real time is critical.

To ensure smooth and reliable operation across a variety of hardware configurations, the system will be optimized for performance. This includes techniques such as dynamic resizing of video frames and multithreaded processing to handle large data streams efficiently without overwhelming the system. By focusing on low computational requirements, the proposed system aims to work effectively even on devices without high-end GPUs. The goal is to create a flexible, user-friendly solution that can be scaled and adapted to a wide range of applications, from autonomous vehicles to safety monitoring and assistive technologies.

3.3 Advantages

- One of the biggest advantages of your system is its real-time performance. Thanks to the power of the YOLOv8 model, your system can detect multiple objects instantly while simultaneously estimating their distance from the camera. This real-time ability is crucial for applications where split-second decisions are important, such as in autonomous vehicles, robotics, or even assistive technologies for the visually impaired.
- Another strong advantage is the system's accessibility and cost-effectiveness. Unlike traditional systems that rely on expensive sensors like LIDAR, your project uses standard cameras and smart algorithms to estimate distances, making the solution much more affordable and easier to implement. This opens the door for widespread adoption in areas like public safety, smart city projects, and personal assistive devices without the heavy burden of high hardware costs.
- Additionally, your project offers multi-modal feedback, combining visual cues (bounding boxes and distance displays) with audio alerts using text-to-speech. This makes it incredibly helpful for users with different needs, including people with visual impairments, by offering intuitive and immediate feedback. Plus, the system is designed to be lightweight and hardware-efficient, meaning it can run smoothly even on devices with lower computational power, without the need for high-end GPUs. This ensures flexibility, scalability, and broad usability across many industries and communities.
- Another major advantage of your system is its flexibility across different input types. Whether it's a live webcam feed, a recorded video, or even a static image, your system can adapt and process them seamlessly. This flexibility means it can be used in a variety of real-world scenarios from real-time traffic monitoring to analyzing CCTV footage or even personal mobility assistance.
- The project also shines in terms of safety enhancement. By accurately detecting objects and alerting users about nearby obstacles, your system significantly reduces the risk of accidents. In industrial environments, for example, it can warn workers about approaching machinery, helping prevent workplace injuries. In everyday life, it can assist pedestrians, cyclists, or visually impaired individuals to move more safely through their environment.
- Scalability is another key strength. Since the system doesn't heavily depend on specialized hardware, it can easily scale across different platforms from small handheld devices like smartphones to larger systems used in cars, drones, or security stations. You can upgrade or expand the system without needing to redesign it from scratch, which is a big advantage for future-proofing your project.
- Lastly, the system's potential for integration with other technologies is a strong advantage. It could easily be combined with GPS, IoT devices, autonomous systems, or cloud-based analytics to create even smarter solutions. This makes your project future-ready a strong foundation that can evolve into even more complex and impactful applications over time.

3.4 Feasibility Study

A feasibility study for the Real-Time Object Detection and Distance Mapping project involves assessing several critical aspects to determine the project's viability, including technical, economic, operational, and legal feasibility.

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

3.4.1 Economic Feasibility

From an economic standpoint, the project is highly feasible. Since the system is designed to work with standard cameras and commodity hardware, there's no need for expensive specialized sensors like LIDAR or high-end GPUs. This significantly reduces the initial investment and ongoing maintenance costs. The use of an efficient model like YOLOv8, which balances performance with low computational demand, ensures that even moderate hardware can support the system without needing upgrades. Additionally, because it can be deployed across multiple sectors — such as transportation, safety, healthcare, and smart cities — the potential return on investment (ROI) is high. This makes the project a cost-effective and sustainable solution for both individual users and organizations.

3.4.2 Technical Feasibility

Technically, the project is very feasible with current technologies. YOLOv8 is a proven, reliable object detection model that provides high accuracy while maintaining real-time performance. Distance estimation techniques using focal length and pixel scaling are well-established, ensuring that accurate distance mapping is achievable without complex or unstable methods. Furthermore, by incorporating performance optimization techniques like dynamic frame resizing and multithreading, the system can maintain smooth functionality even on systems with limited processing capabilities. The integration of visual feedback (bounding boxes) and auditory feedback (text-to-speech) can be implemented with existing libraries, making development straightforward. Overall, there are no significant technical barriers that would prevent the project's successful implementation.

3.4.3 Social Feasibility

Socially, the project is highly beneficial and positively feasible. It directly addresses important needs, such as improving accessibility for visually impaired individuals and enhancing public and workplace safety. By providing real-time alerts and environmental awareness, the system promotes inclusivity, independence, and safety for a wide range of users. Moreover, the system aligns well with broader societal trends toward smarter, safer, and more connected environments, such as smart cities and intelligent transportation systems. The adoption of such technology can also raise awareness and encourage further innovation in assistive technologies, making it socially acceptable and well-supported by communities and organizations focused on health, safety, and accessibility.

3.5 System Specification

The system specifications for the Real-Time Object Detection and Distance Mapping project detail the essential software requirements needed for its successful implementation.

3.5.1 Software Specification

- **Operating System:** Windows 10/11, Linux (Ubuntu 18.04 or later), or macOS (Latest versions preferred)
- **Programming Language:** Python 3.8 or above
- **Libraries and Frameworks:**
    - OpenCV (for video and image processing)
    - PyTorch or TensorFlow (for deep learning and running YOLOv8)
    - YOLOv8 (Ultralytics repository)
    - Text-to-Speech library (like pyttsx3 or gTTS)
    - NumPy (for mathematical operations)
    - Additional Python packages: matplotlib.
- **Other Tools:**
- **IDE:** VS Code, PyCharm, or Jupyter Notebook for development
- **Version Control:** Git (for tracking code versions)
- **Virtual Environment:** (Optional) Anaconda or venv for managing dependencies easily

CHAPTER 4

MODULE DESCRIPTION

4.1 General Architecture

This diagram represents a workflow for classifying Real-Time Object Detection and Distance Mapping.

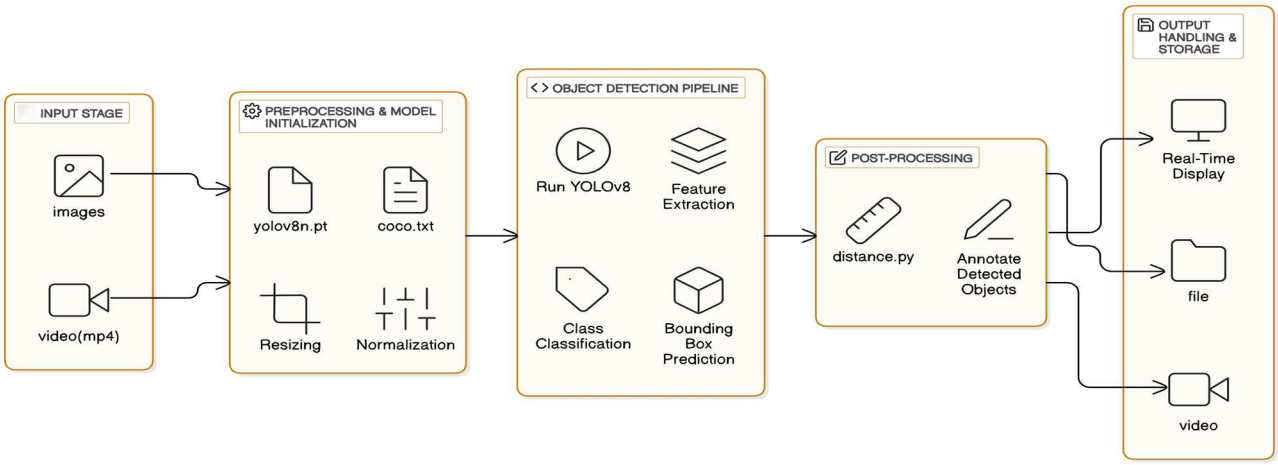


Figure 4.1.1 General Architecture

The architecture of the Real-Time Object Detection and Distance Mapping system is designed to be simple, powerful, and flexible allowing it to work with both images and video inputs. The whole system flows through several well-organized stages to make real-time detection and distance estimation possible.

4.2 Data Flow Diagram

1. Input Stage

Everything begins at the Input Stage. Here, users can provide either static images (like photos) or dynamic videos (such as surveillance footage or webcam streams). This flexibility means the system can be used in a variety of environments whether it's analysing traffic videos, assisting visually impaired individuals in real-time, or simply scanning a folder of images for research purposes.

2. Preprocessing and Model Initialization

- Once the input is given, the system moves into Preprocessing. Before any object detection happens, the data needs to be prepared properly:
- The pre-trained YOLOv8 model (yolov8n.pt) is loaded — this model knows how to recognize a wide range of objects from prior training.
- Along with the model, a COCO label file (coco.txt) is loaded, listing all the object categories YOLO can detect (like person, bicycle, car, etc.).
- The input images or video frames are resized to a size YOLO expects, making the detection process faster and more accurate.
- Normalization is applied, where pixel values are scaled to a smaller range to help the neural network work more efficiently without being overwhelmed by big numbers.
- This step ensures that the system always works with clean, standardized data — regardless of the quality or size of the original input.

3. Object Detection Pipeline

- Now the main heart of the system kicks in: the Object Detection Pipeline.
- The pre-processed frames are fed into the YOLOv8 model.
- Feature extraction happens YOLO analyzes the images and detects patterns, edges, and colors that hint at different objects.
- Next is classification, where each detected object is given a label for example, "person", "dog", "car", etc.
- Bounding boxes are drawn around each detected object, showing exactly where it is located in the frame.
- This pipeline happens very fast, almost instantly, allowing the system to work in real-time.

4. Post-Processing

- Detection alone is not enough we need to know how far away each object is. That's where Post-Processing comes in:
- Using a separate script (video with distance.py), the system estimates the physical distance of each detected object from the camera.
- It relies on simple geometric relationships, using the known focal length of the camera and the size of the object in the image to calculate real-world distance.
- Then, the system annotates each detected object with this distance information, making it easy to understand both what the object is and how far it is.
- This step transforms basic object detection into a much smarter understanding of the environment.

5. Output Handling and Storage

- After detection and distance mapping, the system offers different output options:
- If immediate feedback is needed, the system supports Real-Time Display showing the results live with bounding boxes, object names, and distance labels.
- If a record is needed, the system can save the results into files (like annotated images).
- For video inputs, it can save the processed video with all annotations intact, creating a permanent record of detections and distances over time.
- This flexibility ensures that the system is useful for everything from live monitoring to offline analysis.

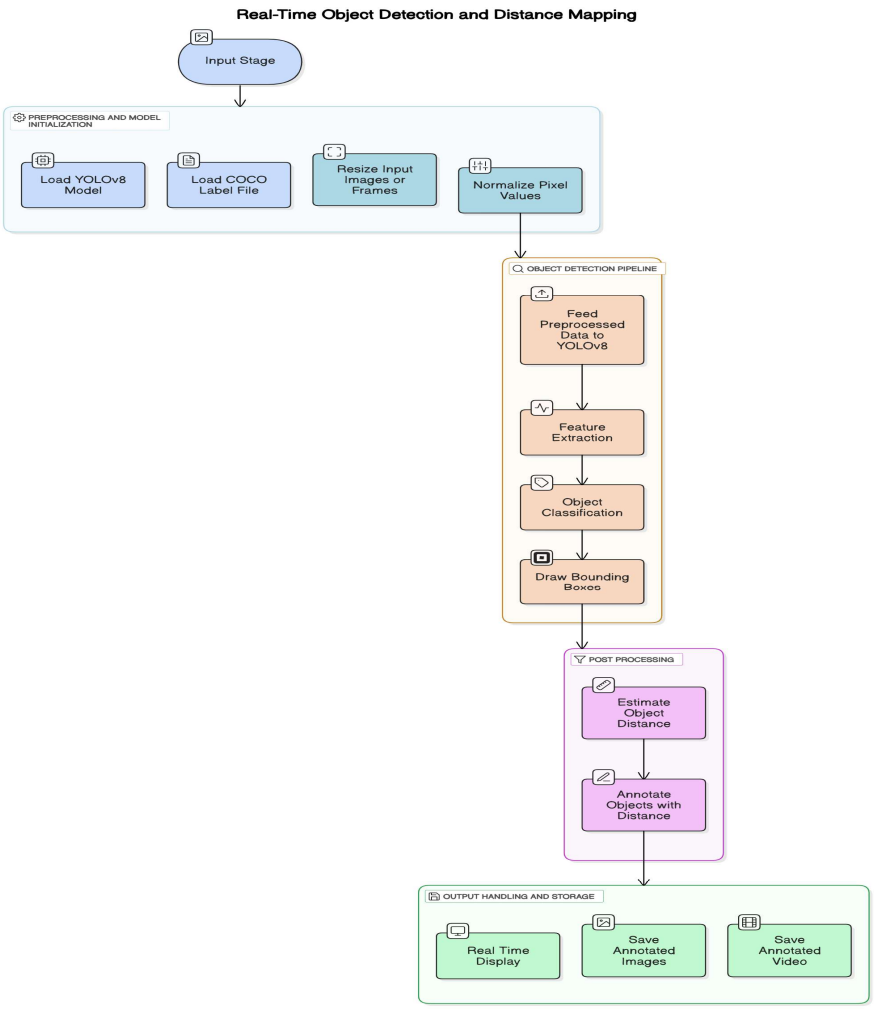


Figure 4.2.1 Data Flow Diagram

4.3 Module Description

1. Input Handling Module

The first point of contact between the system and the outside world happens here. This module is designed to smoothly handle different types of input, whether it's static images like JPEGs or PNGs, or dynamic video streams like MP4 files or live camera feeds. No matter if it's just a single photo or an ongoing video feed, this part of the system makes sure everything is correctly formatted and ready for the next steps. It's like opening the door for the visual data to enter the system.

2. Preprocessing and Model Initialization Module

Before the AI can start detecting objects, the input data needs some preparation. This module takes care of that. It loads the trained YOLOv8 model, imports the object labels (like "person", "car", "bicycle" from the COCO dataset), and resizes the images or video frames to match the model's input size, usually 640x640 pixels. It also normalizes the pixel values, shrinking them to a range between 0 and 1 for faster and more stable processing. In simple terms, this stage ensures the input is neat, clean, and ready for high-speed detection.

3. Object Detection Module

This is the real "brain" of the system where the action happens. Using the powerful YOLOv8 model, this module detects objects in real time. It identifies features, classifies the detected objects, and draws bounding boxes around them. Each object is also given a confidence score, showing how sure the model is about what it sees. In short, this module transforms raw images and videos into detailed, understandable information — letting us know what's in the scene and exactly where.

4. Distance Mapping Module

After detecting objects, it's important to know how far away they are — and that's where this module comes in. It calculates the distance between the camera and each detected object using smart geometric methods, like pixel-to-meter scaling. For instance, it might use known average sizes of objects (like a person's height) to estimate how close or far something is. Thanks to this module, the system doesn't just recognize objects — it also understands their spatial relationship to the observer.

5. Post-Processing Module

Once detection and distance estimation are done, the system needs to present the results in a clean and helpful way and that's the job of this module. It annotates images and video frames by adding bounding boxes, labels, and distance information neatly on the display. If needed, it can also generate voice alerts using text-to-speech technology, which is especially useful for assistive applications. This module makes sure the output looks polished, organized, and is easy for users to understand.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Training and Testing Results:

The table summarizes the performance of models on Real-Time Object Detection and Distance Mapping.

Table 5.1 Training and Validation Results

Object	Confidence
Person	0.91
Bus	0.91
Person	0.84
Person	0.83
Person	0.78
Backpack	0.71
Person	0.91
HandBag	0.66
Truck	0.68
Bus	0.75
HandBag	0.66

The object detection system underwent evaluation using various images and videos with the YOLOv8 models (yolov8m.pt and yolov8n.pt). These models effectively recognized and labeled objects, enclosing them within bounding boxes. The detection accuracy was notably high for static images, while the system demonstrated reliable tracking capabilities in video sequences. The real-time detection module operated efficiently, exhibiting minimal lag, and the video with distance.py script provided additional functionality by estimating distances to objects, thereby enhancing spatial awareness.

To evaluate the system's accuracy, the image "bus2.jpg" (448 × 640 pixels) was scrutinized. The model successfully detected several objects:

- People: Five individuals were identified, with confidence levels ranging from 0.39 to 0.91. The lower confidence scores indicate that some individuals may have been partially obscured.
- Buses: Two buses were detected, one with a high confidence score of 0.91 and the other with a low score of 0.27, suggesting potential misclassification.
- Truck: A truck was identified, but with a low confidence score of 0.27, indicating uncertainty in its classification.
- Accessories: A backpack was recognized with a confidence score of 0.71, along with two handbags with scores of 0.38 and 0.25, respectively, reflecting some inconsistency in classification.

The total processing time for this image was 98.5 milliseconds, with 92.2 milliseconds allocated to inference. These findings underscore the system's efficiency in real-time applications, although the low-confidence detections highlight the necessity for refining confidence thresholds. The performance of the object detection system was influenced by factors such as model selection, image resolution, and environmental conditions. The yolov8m.pt model provided higher accuracy, making it ideal for applications that prioritize precision, while the yolov8n.pt model offered faster performance, albeit with a slight compromise in accuracy.

Several challenges were identified, particularly the model's sensitivity to low-light conditions, which occasionally led to missed detections or lower confidence scores. Implementing image processing techniques, such as brightness adjustments or histogram equalization, may enhance performance. Furthermore, the model sometimes misclassified or overlooked smaller objects when they overlapped. Employing methods like non-maximum suppression (NMS) and object tracking could alleviate these problems.

In conclusion, the YOLOv8 model shows significant promise for real-time object detection. Future enhancements could involve fine-tuning the model with tailored datasets and incorporating cloud-based or edge computing solutions to improve scalability. By addressing these limitations, the system's reliability across various applications could be further strengthened.

5.2 Classification Report

Ground truth refers to the actual labels for each object in an image, which are typically manually annotated and represent the true class and location of each object. Your detection script provides predicted classes (e.g., "person," "bus," or "truck") along with confidence scores and bounding boxes. To compare these predictions to the ground truth, you can use a metric like Intersection over Union (IoU), which measures how well the predicted bounding boxes match the true ones. If the IoU exceeds a certain threshold (e.g., 0.5), the prediction is considered a True Positive (TP). If it's incorrect, it's a False Positive (FP), and if an object is missing, it's a False Negative (FN).

5.3 Detection

The object detection script successfully identified multiple objects in the image, including five persons, two buses, one truck, one backpack, and two handbags, with varying confidence levels and distances. The detected persons were recognized with high confidence (up to 0.91) and distances ranging from 3.52m to 26.14m, while the trucks and buses had lower confidence scores (around 0.27). Bounding box coordinates were provided for each object (in the format [x_min, y_min, x_max, y_max]), with some objects, like the backpack and handbags, lacking distance information. This indicates that the detection model performed well on certain objects but was less confident with others, particularly at longer distances.

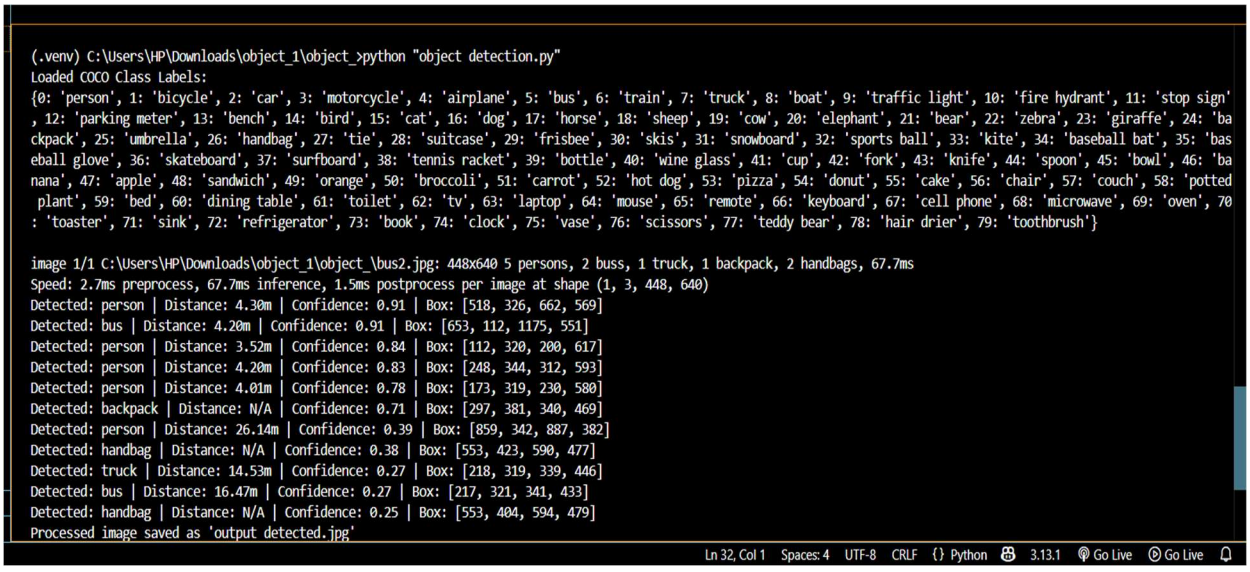


Figure 5.3 Object Detection Results

CHAPTER 6

CONCLUSION AND FUTURE ENCHANCEMENTS

The Real-Time Object Detection and Distance Mapping project brings together fast object detection and smart distance estimation to create a powerful real-time vision system. Using YOLOv8, the system can quickly and accurately detect objects from live video streams or static images, while also calculating how far those objects are from the camera. This dual capability opens up opportunities for safer navigation, better assistive technologies, and smarter industrial monitoring. By providing both visual and audio feedback, the system makes environments more accessible and safer, particularly for visually impaired individuals or in high-risk workplaces. Overall, this project successfully builds a strong foundation for creating intelligent, real-world applications that combine speed, accuracy, and practicality.

Looking ahead, there are several exciting ways this system could grow even further. Improvements like integrating depth sensors could make distance measurements even more accurate, and adding multi-object tracking would allow the system to understand movement and behaviour over time. Optimizing the model for edge devices like Raspberry Pi could make it portable and energy-efficient, allowing it to be used in more flexible environments. There's also potential to expand into full 3D mapping and navigation systems for robots or drones. Smarter feedback mechanisms, like alerts that change depending on object proximity, could make it even more useful for safety and assistive technologies. With these enhancements, the project could evolve into a cutting-edge platform for future smart vision systems.

REFERENCES

[1] Appiah, Emmanuel Owusu, and Solomon Mensah. "Object detection in adverse weather condition for autonomous vehicles." Multimedia Tools and Applications 83, no. 9 (2024): 28235-28261.

[2] Yeong, De Jong, Gustavo Velasco-Hernandez, John Barry, and Joseph Walsh. "Sensor and sensor fusion technology in autonomous vehicles: A review." Sensors 21, no. 6 (2021): 2140.

[3] Sarasa-Cabezuelo, Antonio. "Prediction of rainfall in Australia using machine learning." Information 13, no. 4 (2022): 163.

[4] Wason, Ritika, Parul Arora, Vishal Jain, Devansh Arora, and M. N. Hoda. "Exploring the Convergence of Artificial Intelligence and Mechatronics in Autonomous Driving." Computational Intelligent Techniques in Mechatronics (2024): 297-316.

[5] Ghintab, Shahad S., and Mohammed Y. Hassan. "Localization for self-driving vehicles based on deep learning networks and RGB cameras." International Journal of Advanced Technology and Engineering Exploration 10, no. 105 (2023): 1016.

[6] Hasanujjaman, Muhammad, Mostafa Zaman Chowdhury, and Yeong Min Jang. "Sensor fusion in autonomous vehicle with traffic surveillance camera system: detection, localization, and AI networking." Sensors 23, no. 6 (2023): 3335.

[7] Hacohen, Shlomi, Oded Medina, and Shraga Shoval. "Autonomous driving: A survey of technological gaps using google scholar and web of science trend analysis." IEEE Transactions on Intelligent Transportation Systems 23, no. 11 (2022): 21241-21258.

[8] Mahima, KT Yasas, Asanka G. Perera, Sreenatha Anavatti, and Matt Garratt. "Toward robust 3d perception for autonomous vehicles: A review of adversarial attacks and countermeasures." IEEE Transactions on Intelligent Transportation Systems (2024).

[9] Huang, Kun, Yifu Wang, Si'ao Zhang, Zhirui Wang, Zhanpeng Ouyang, Zhenghua Yu, and Laurent Kneip. "OpenGV 2.0: Motion prior-assisted calibration and SLAM with vehicle-mounted surround-view systems." arXiv preprint arXiv:2503.03230 (2025).

[10] Yan, Shen, Maojun Zhang, Yang Peng, Yu Liu, and Hanlin Tan. "Agenti2p: Optimizing image-to-point cloud registration via behaviour cloning and reinforcement learning." Remote Sensing 14, no. 24 (2022): 6301.

APPENDIX A

CODING

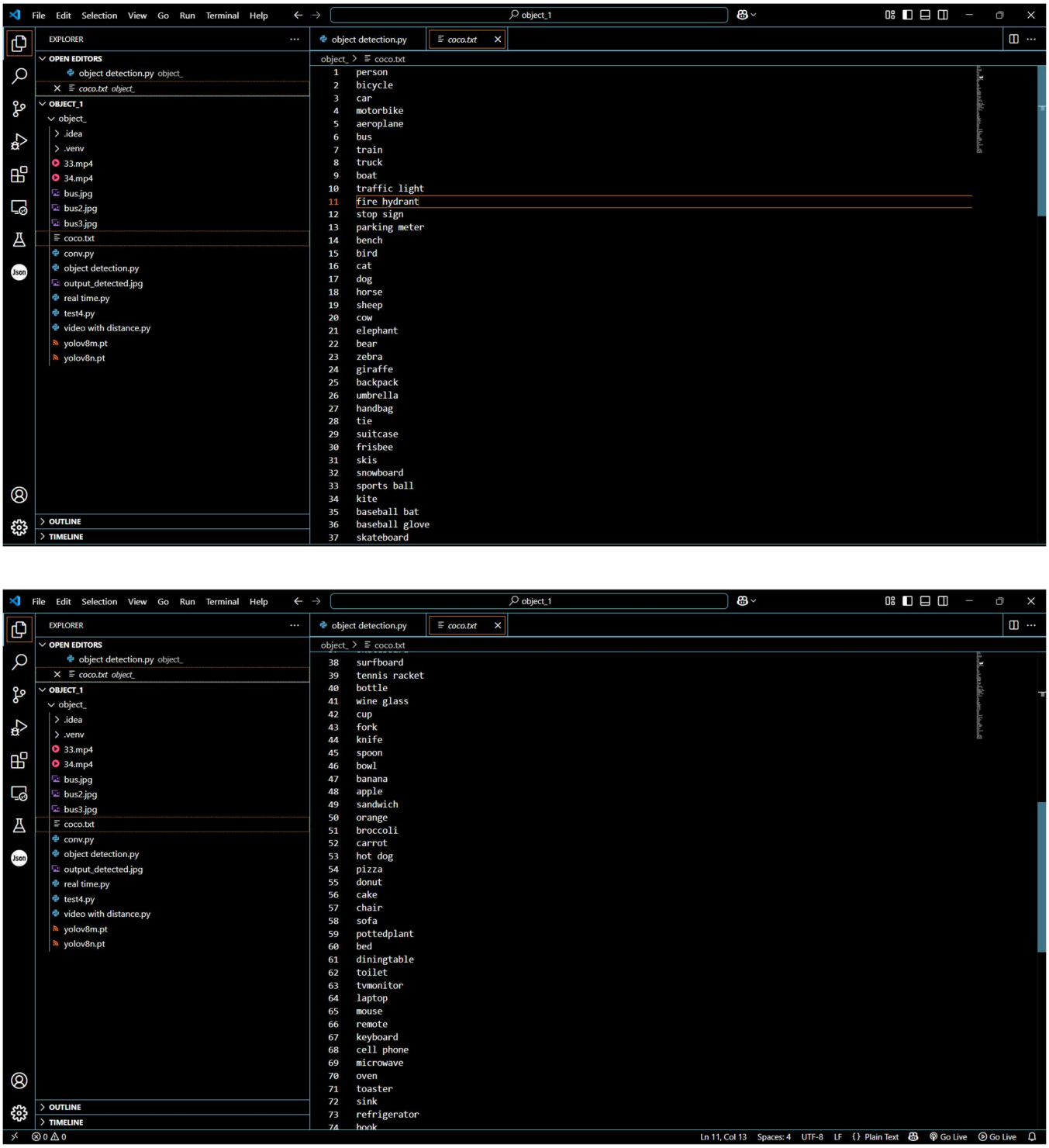
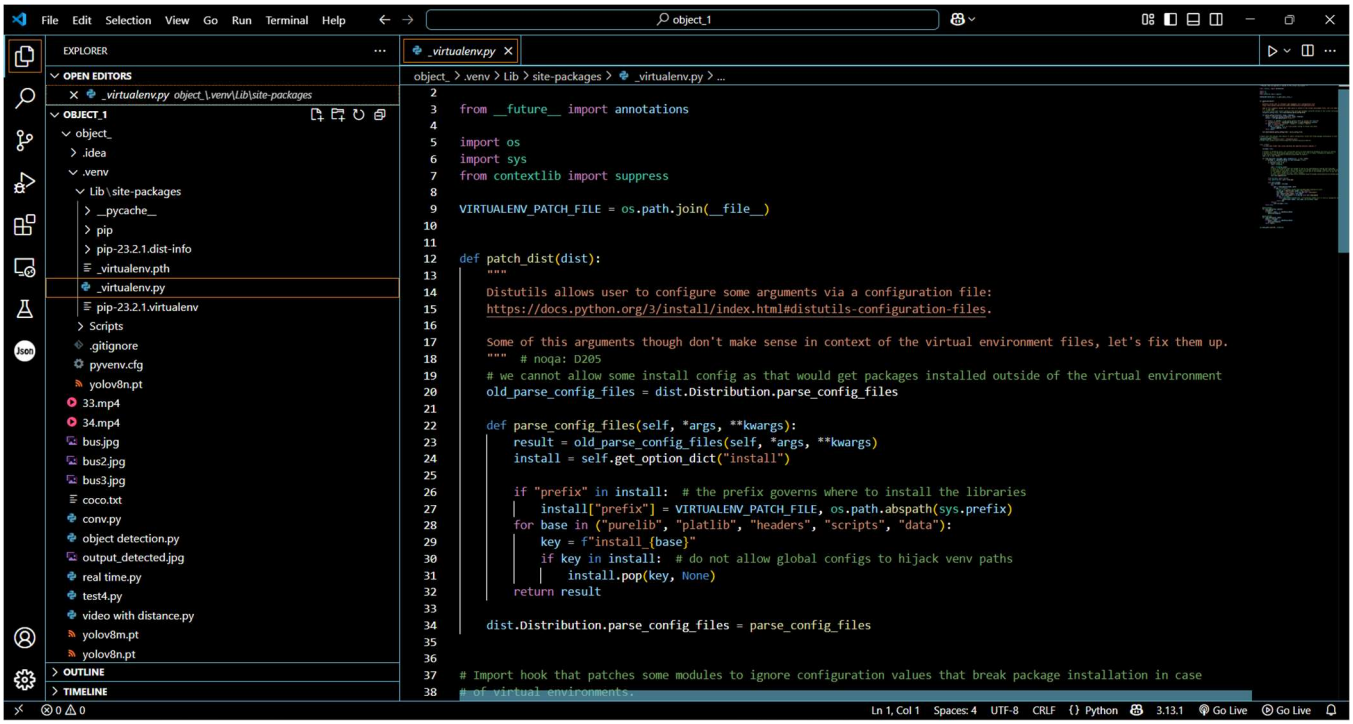


Figure 6.1. Data Set (COCO)



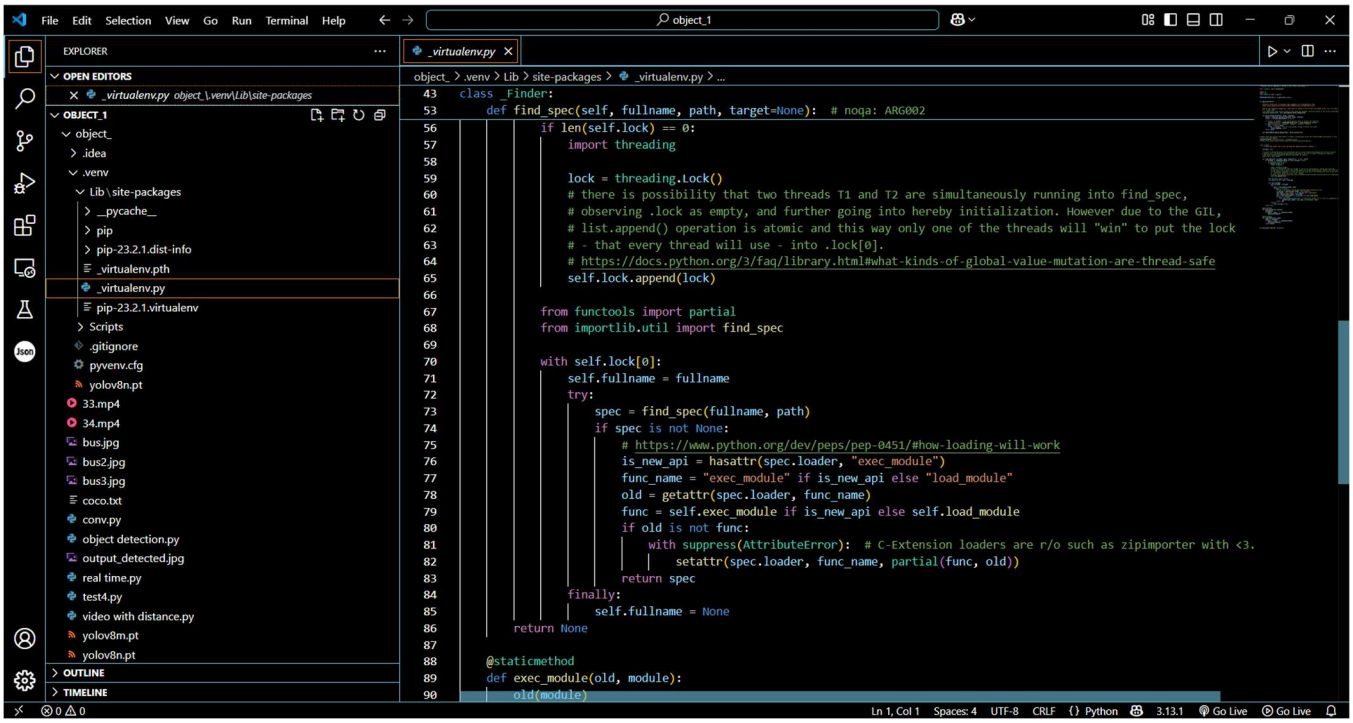
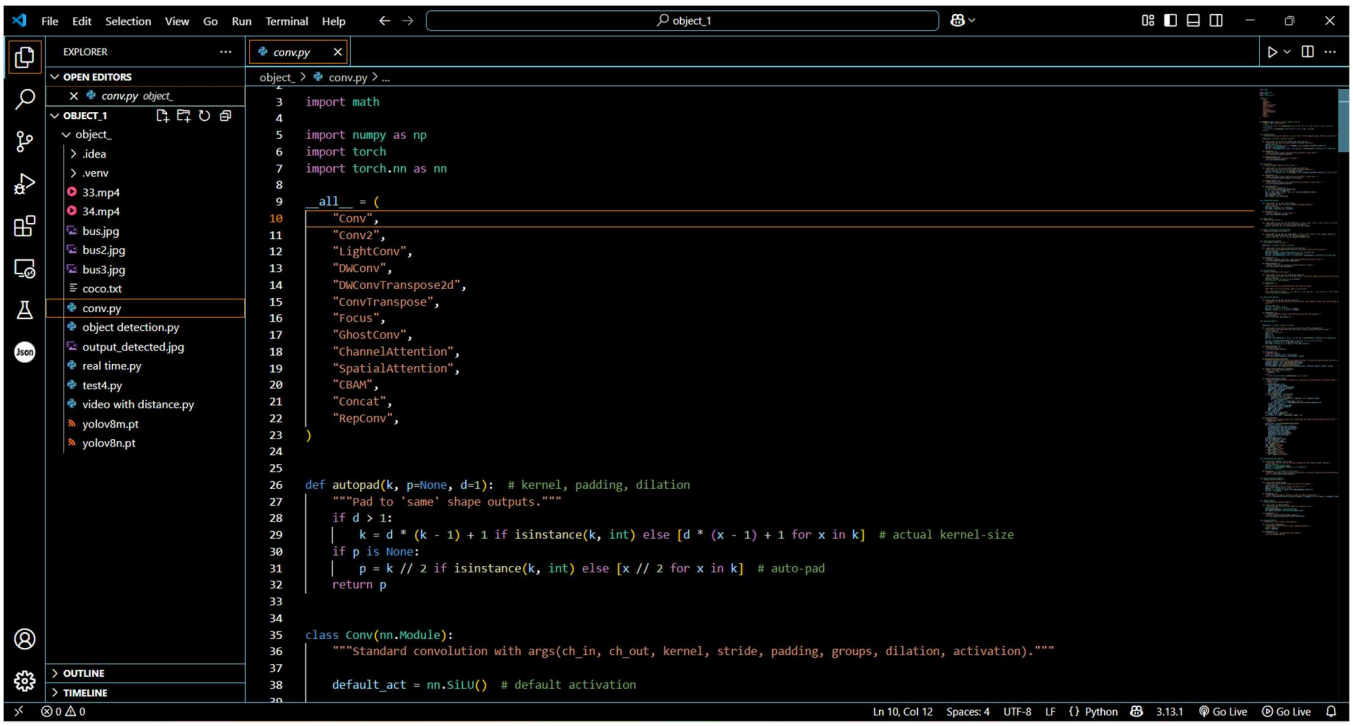
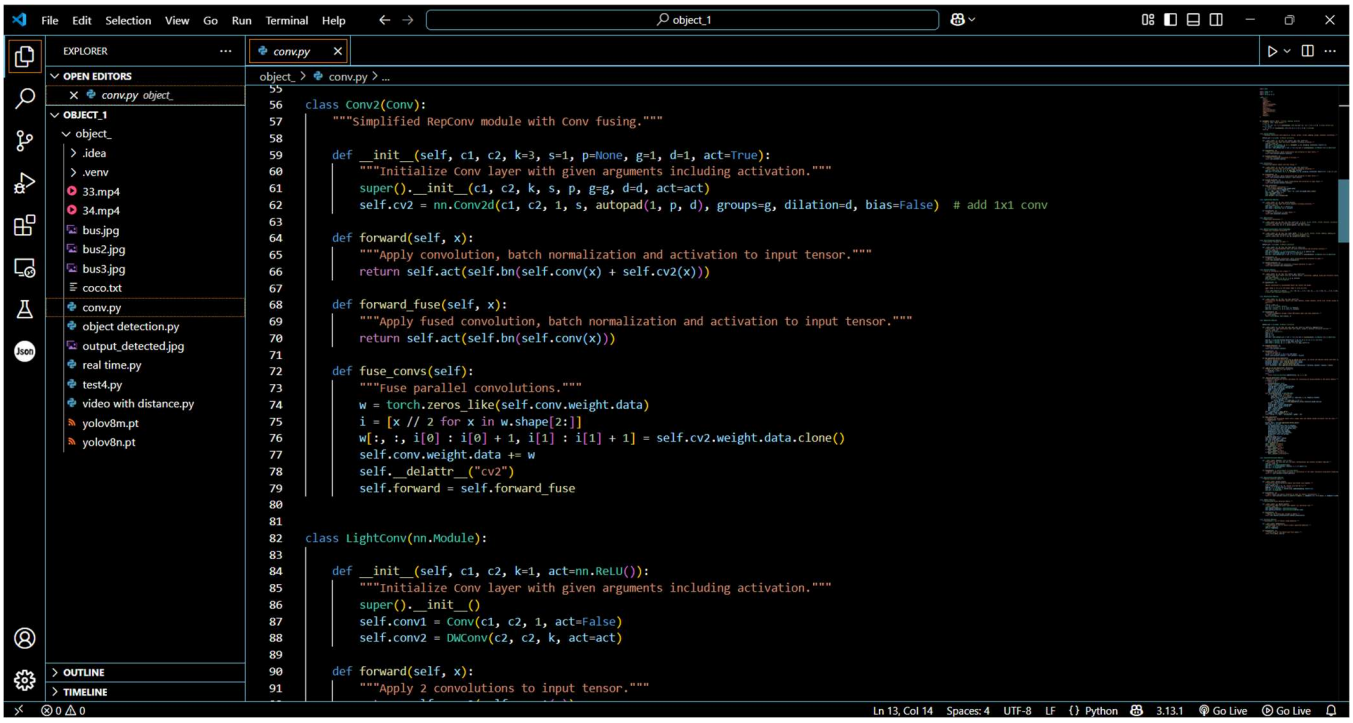
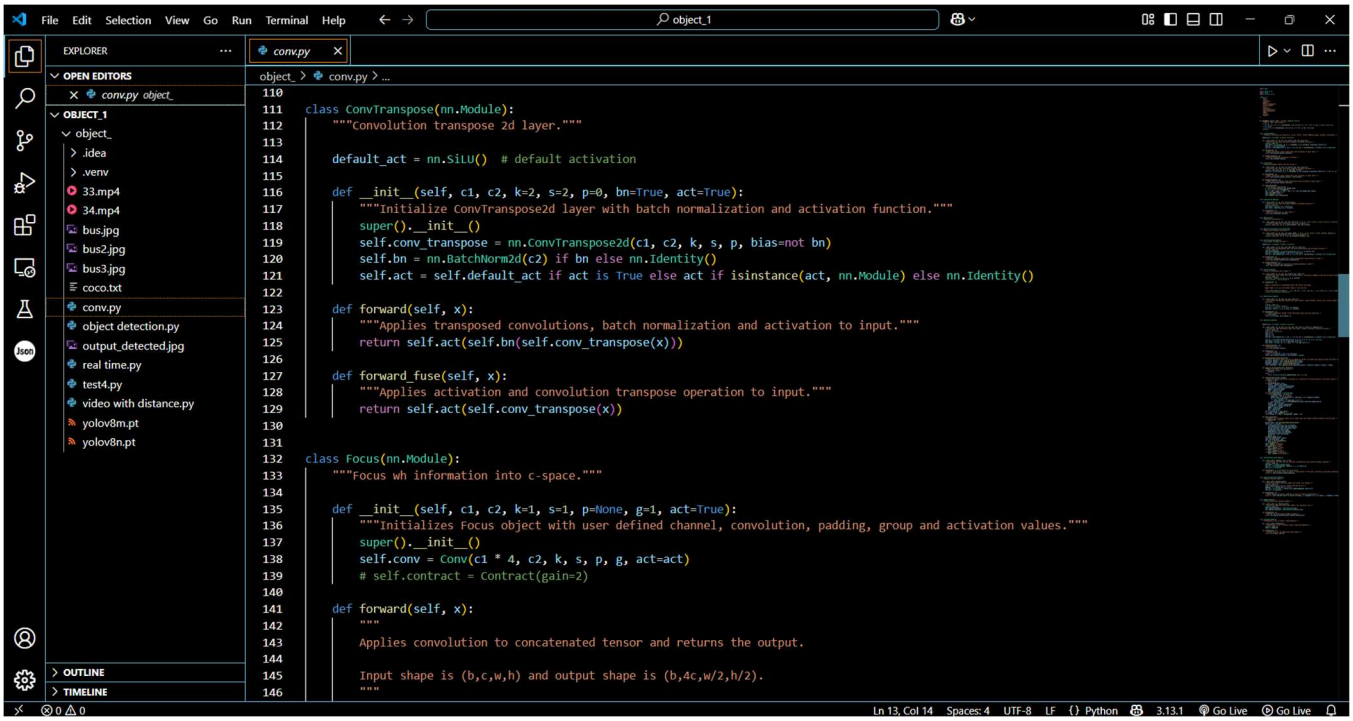


Figure 6.2 Virtual Environment(venv)


File Edit Selection View Go Run Terminal Help
object_1
EXPLORER
conv.py
OPEN EDITORS
conv.py object_
OBJECT_1
object_
.idea
.venv
33.mp4
34.mp4
bus.jpg
bus2.jpg
bus3.jpg
coco.txt
conv.py
object detection.py
output_detected.jpg
real time.py
test4.py
video with distance.py
yolov8m.pt
yolov8n.pt
OUTLINE
TIMELINE
object_ > conv.py > ...
3 import math
4
5 import numpy as np
6 import torch
7 import torch.nn as nn
8
9 __all__ = (
10 "Conv",
11 "Conv2",
12 "LightConv",
13 "DWConv",
14 "DWConvTranspose2d",
15 "ConvTranspose",
16 "Focus",
17 "GhostConv",
18 "ChannelAttention",
19 "SpatialAttention",
20 "CBAM",
21 "Concat",
22 "RepConv",
23)
24
25
26 def autopad(k, p=None, d=1): # kernel, padding, dilation
27 """Pad to 'same' shape outputs."""
28 if d > 1:
29 k = d * (k - 1) + 1 if isinstance(k, int) else [d * (x - 1) + 1 for x in k] # actual kernel-size
30 if p is None:
31 p = k // 2 if isinstance(k, int) else [x // 2 for x in k] # auto-pad
32 return p
33
34
35 class Conv(nn.Module):
36 """Standard convolution with args(ch_in, ch_out, kernel, stride, padding, groups, dilation, activation)."""
37
38 default_act = nn.SiLU() # default activation
Ln 10, Col 12 Spaces: 4 UTF-8 LF Python 3.13.1 Go Live


File Edit Selection View Go Run Terminal Help
object_1
EXPLORER
conv.py
OPEN EDITORS
conv.py object_
OBJECT_1
object_
.idea
.venv
33.mp4
34.mp4
bus.jpg
bus2.jpg
bus3.jpg
coco.txt
conv.py
object detection.py
output_detected.jpg
real time.py
test4.py
video with distance.py
yolov8m.pt
yolov8n.pt
OUTLINE
TIMELINE
object_ > conv.py > ...
56 class Conv2(Conv):
57 """Simplified RepConv module with Conv fusing."""
58
59 def __init__(self, c1, c2, k=3, s=1, p=None, g=1, d=1, act=True):
60 """Initialize Conv layer with given arguments including activation."""
61 super().__init__(c1, c2, k, s, p, g=g, d=d, act=act)
62 self.cv2 = nn.Conv2d(c1, c2, 1, s, autopad(1, p, d), groups=g, dilation=d, bias=False) # add 1x1 conv
63
64 def forward(self, x):
65 """Apply convolution, batch normalization and activation to input tensor."""
66 return self.act(self.bn(self.conv(x) + self.cv2(x)))
67
68 def forward_fuse(self, x):
69 """Apply fused convolution, batch normalization and activation to input tensor."""
70 return self.act(self.bn(self.conv(x)))
71
72 def fuse_convs(self):
73 """Fuse parallel convolutions."""
74 w = torch.zeros_like(self.conv.weight.data)
75 i = [x // 2 for x in w.shape[2:]]
76 w[:, :, i[0] : i[0] + 1, i[1] : i[1] + 1] = self.cv2.weight.data.clone()
77 self.conv.weight.data += w
78 self.__delattr__("cv2")
79 self.forward = self.forward_fuse
80
81
82 class LightConv(nn.Module):
83
84 def __init__(self, c1, c2, k=1, act=nn.ReLU()):
85 """Initialize Conv layer with given arguments including activation."""
86 super().__init__()
87 self.conv1 = Conv(c1, c2, 1, act=False)
88 self.conv2 = DWConv(c2, c2, k, act=act)
89
90 def forward(self, x):
91 """Apply 2 convolutions to input tensor."""
Ln 13, Col 14 Spaces: 4 UTF-8 LF Python 3.13.1 Go Live



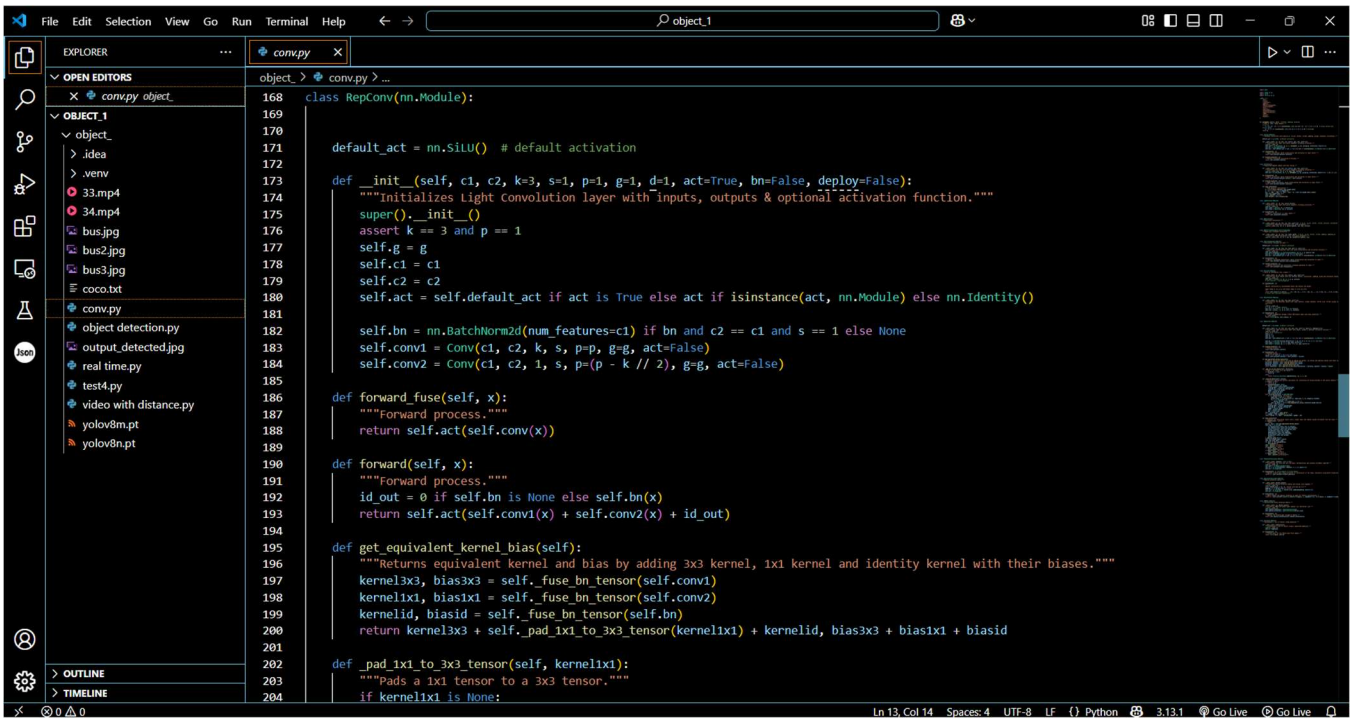
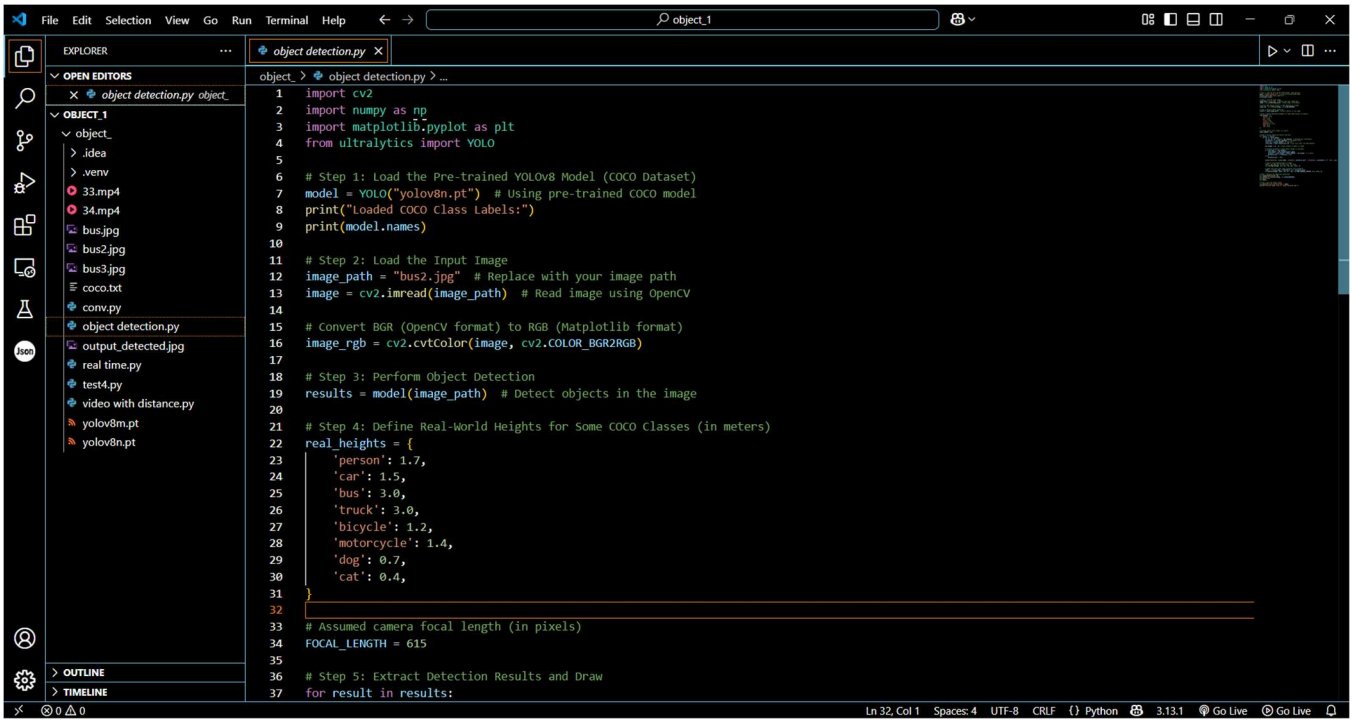
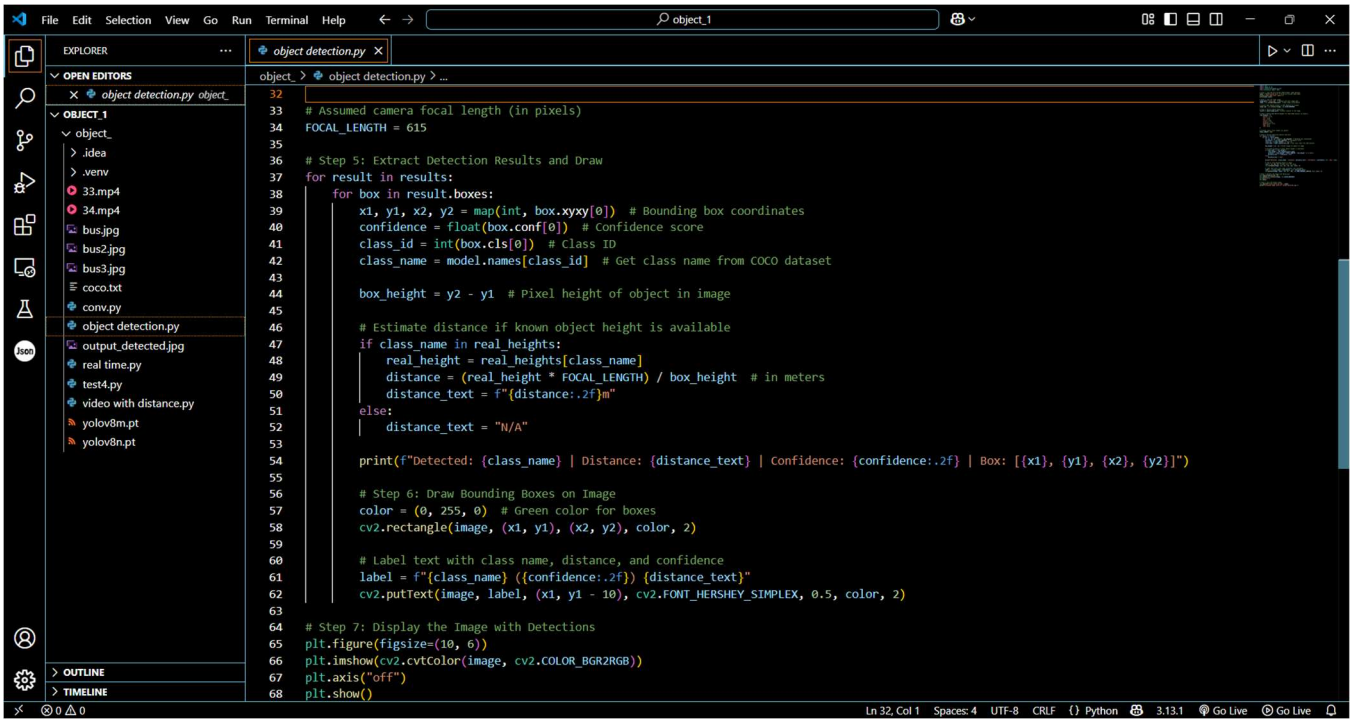


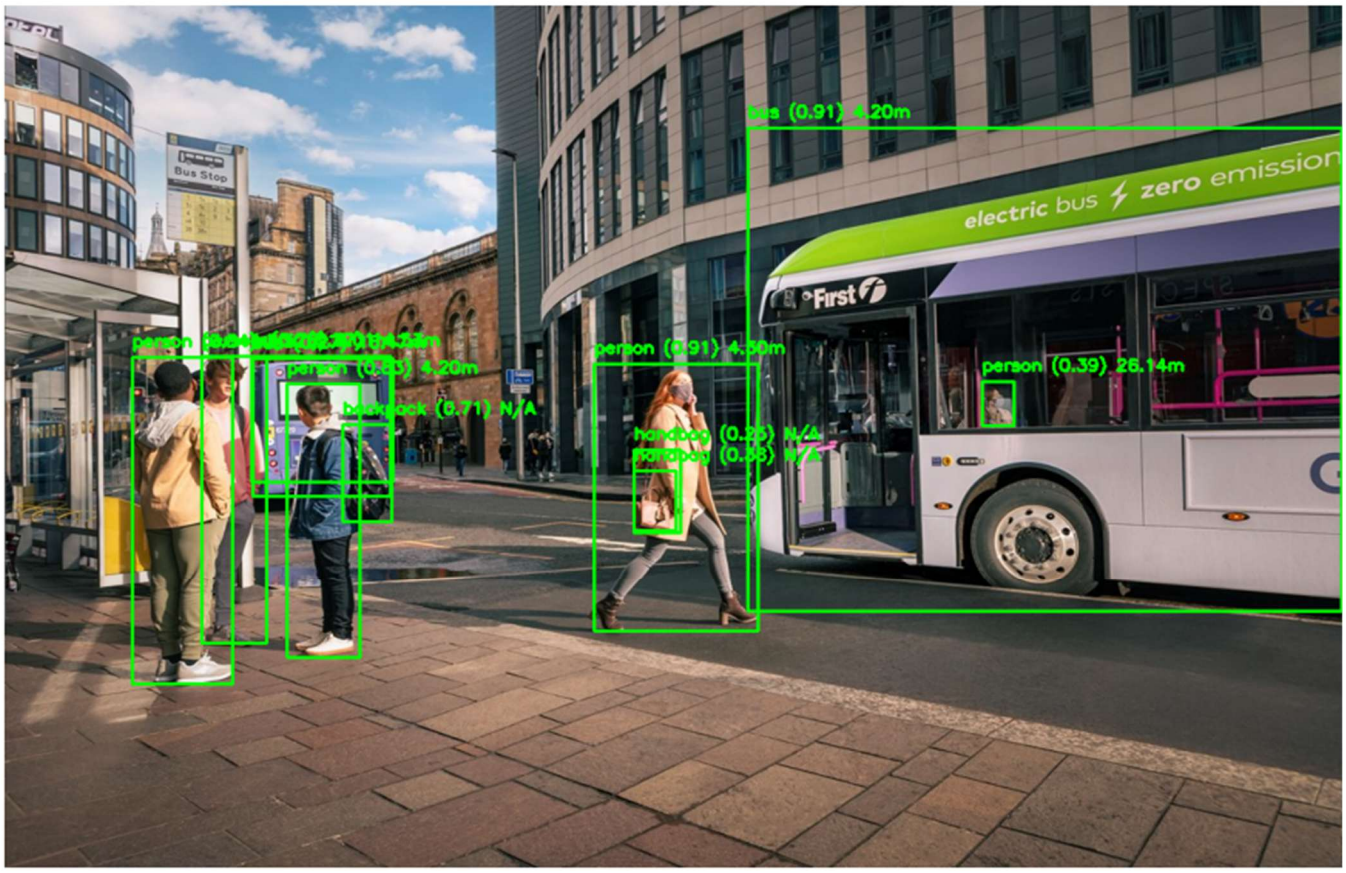
Figure 6.3 neural network layers module(conv)



```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO

# Step 1: Load the Pre-trained YOLOv8 Model (COCO Dataset)
model = YOLO("yolov8n.pt")  # Using pre-trained COCO model
print("Loaded COCO Class Labels:")
print(model.names)

# Step 2: Load the Input Image
image_path = "bus2.jpg"  # Replace with your image path
image = cv2.imread(image_path)  # Read image using OpenCV

# Convert BGR (OpenCV format) to RGB (Matplotlib format)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Step 3: Perform Object Detection
results = model(image_path)  # Detect objects in the image

# Step 4: Define Real-World Heights for Some COCO Classes (in meters)
real_heights = {
    'person': 1.7,
    'car': 1.5,
    'bus': 3.0,
    'truck': 3.0,
    'bicycle': 1.2,
    'motorcycle': 1.4,
    'dog': 0.7,
    'cat': 0.4,
}

# Assumed camera focal length (in pixels)
FOCAL_LENGTH = 615

# Step 5: Extract Detection Results and Draw
for result in results:
```



```python
# Assumed camera focal length (in pixels)
FOCAL_LENGTH = 615

# Step 5: Extract Detection Results and Draw
for result in results:
    for box in result.boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0])  # Bounding box coordinates
        confidence = float(box.conf[0])  # Confidence score
        class_id = int(box.cls[0])  # Class ID
        class_name = model.names[class_id]  # Get class name from COCO dataset

        box_height = y2 - y1  # Pixel height of object in image

        # Estimate distance if known object height is available
        if class_name in real_heights:
            real_height = real_heights[class_name]
            distance = (real_height * FOCAL_LENGTH) / box_height  # in meters
            distance_text = f"{distance:.2f}m"
        else:
            distance_text = "N/A"

        print(f"Detected: {class_name} | Distance: {distance_text} | Confidence: {confidence:.2f} | Box: [{x1}, {y1}, {x2}, {y2}]")

        # Step 6: Draw Bounding Boxes on Image
        color = (0, 255, 0)  # Green color for boxes
        cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)

        # Label text with class name, distance, and confidence
        label = f"{class_name} ({confidence:.2f}) {distance_text}"
        cv2.putText(image, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

# Step 7: Display the Image with Detections
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



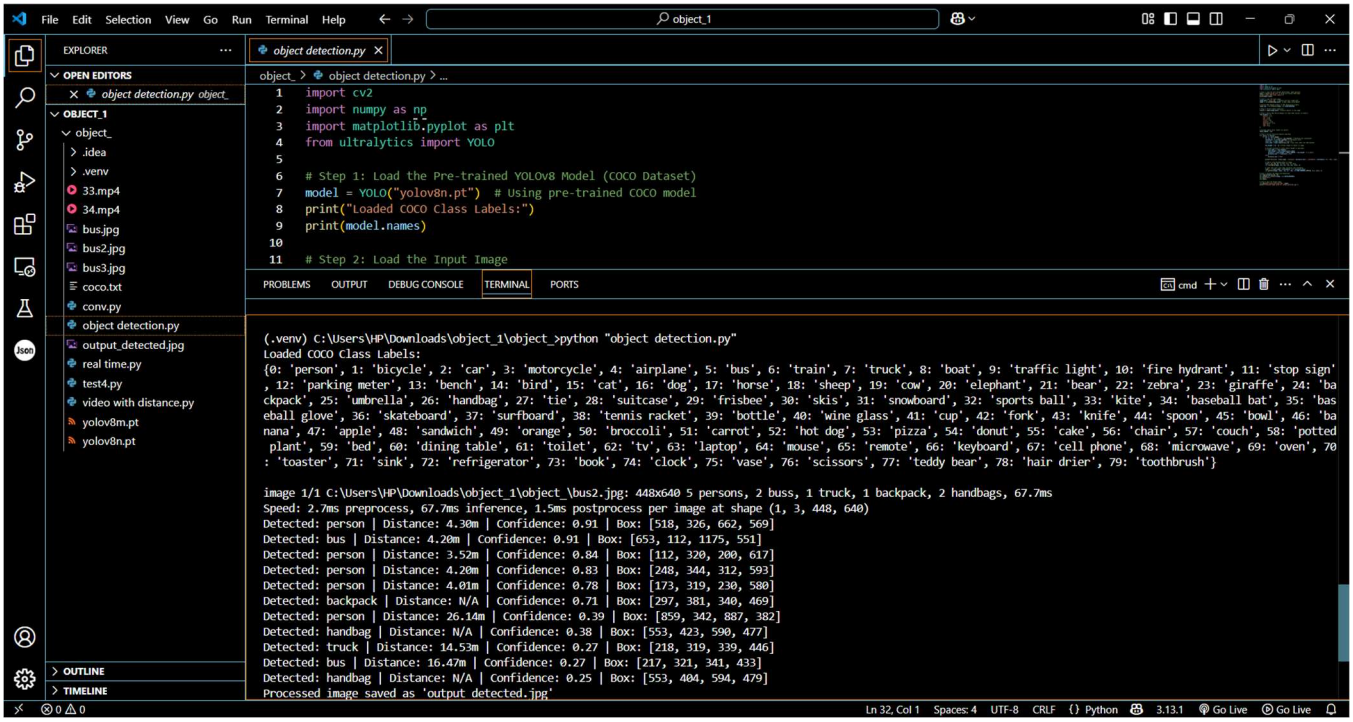
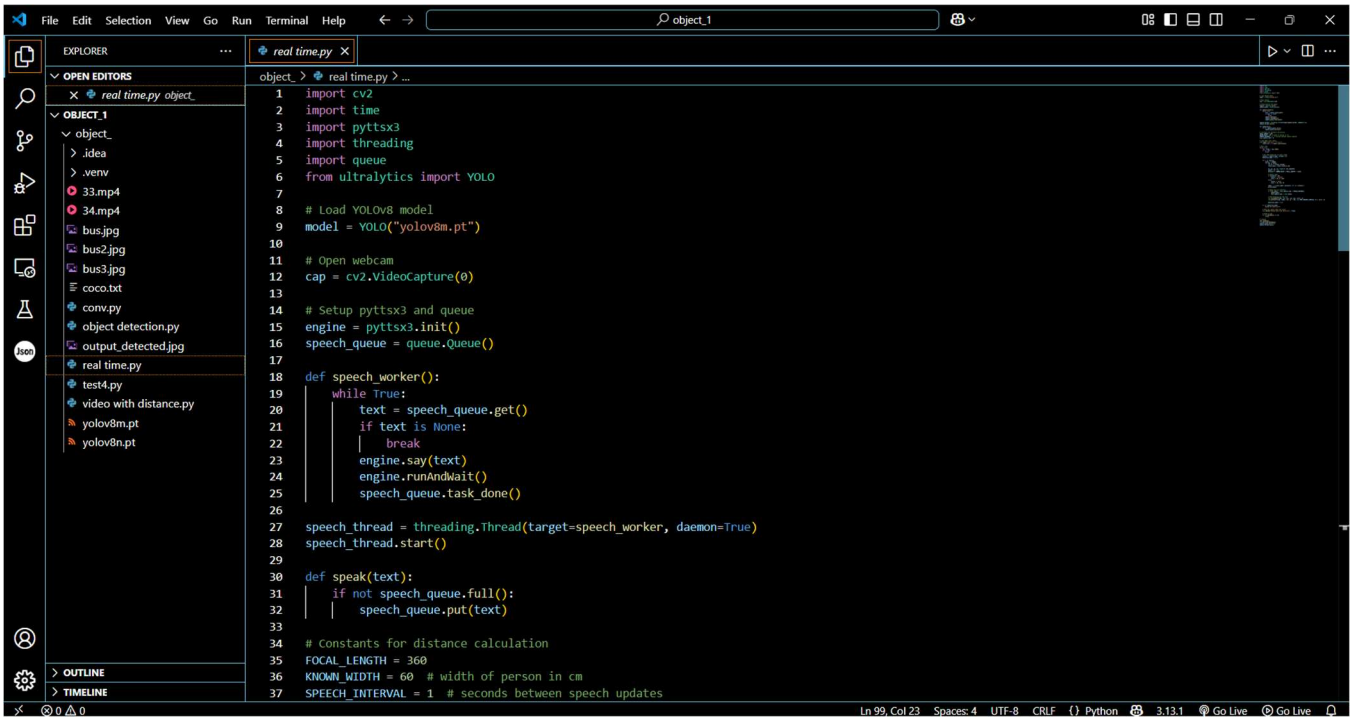
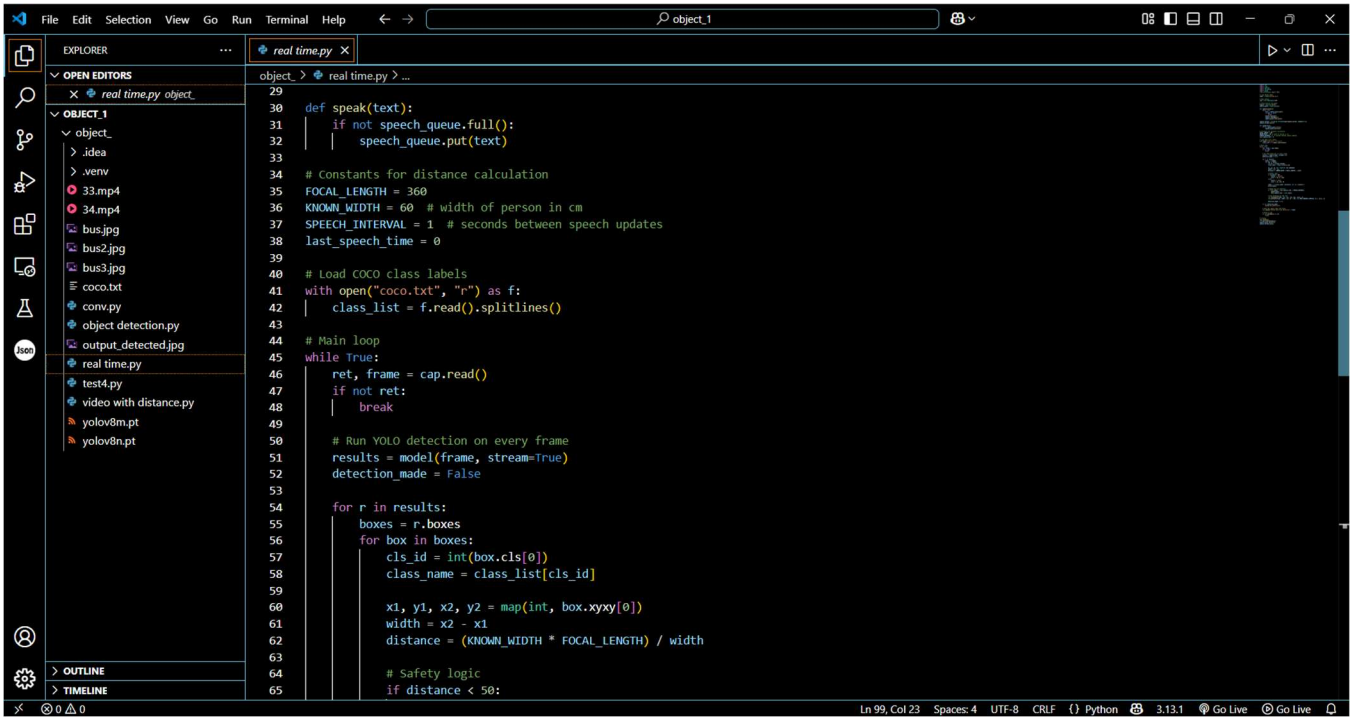
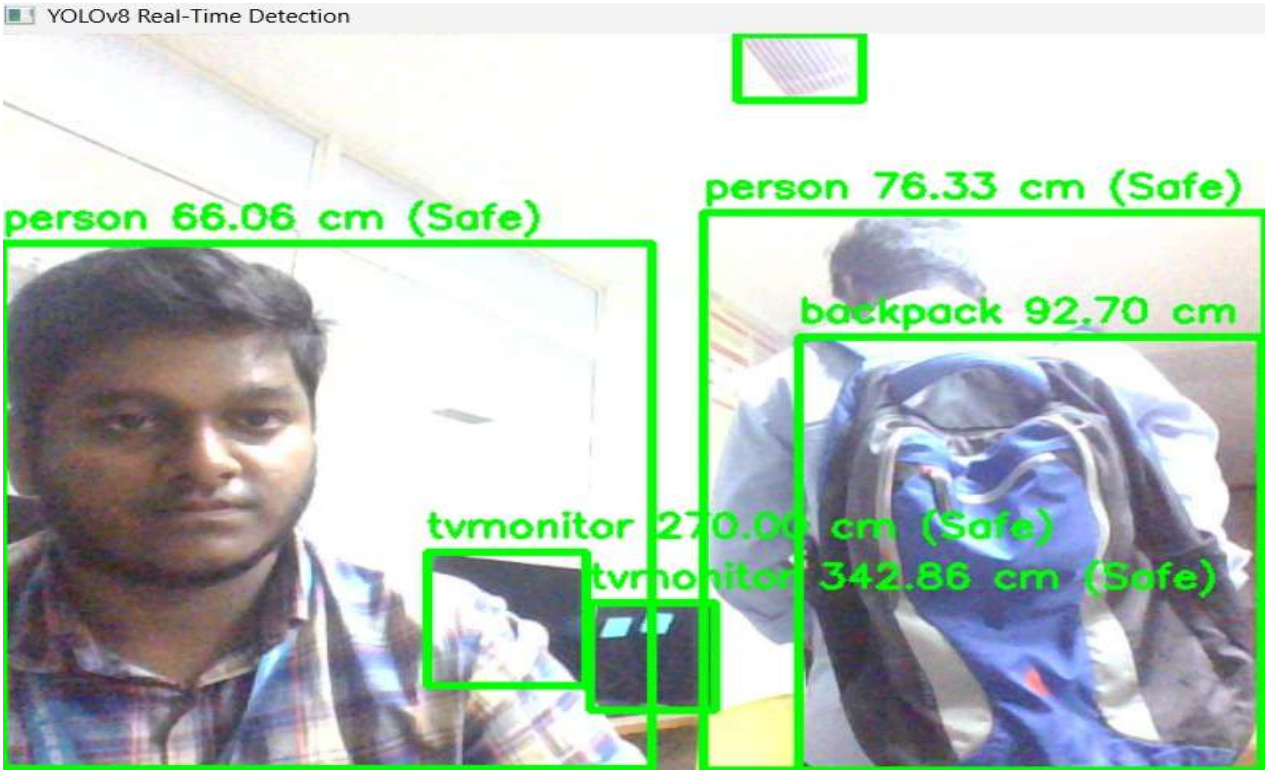


Figure 6.4 Object Detection with Distance Calculation


File Edit Selection View Go Run Terminal Help
object_1
EXPLORER
real time.py
OPEN EDITORS
real time.py object_
OBJECT_1
object_
.idea
.venv
33.mp4
34.mp4
bus.jpg
bus2.jpg
bus3.jpg
coco.txt
conv.py
object detection.py
output_detected.jpg
real time.py
test4.py
video with distance.py
yolov8m.pt
yolov8n.pt
OUTLINE
TIMELINE
object_ > real time.py > ...
1 import cv2
2 import time
3 import pyttsx3
4 import threading
5 import queue
6 from ultralytics import YOLO
7
8 # Load YOLOv8 model
9 model = YOLO("yolov8m.pt")
10
11 # Open webcam
12 cap = cv2.VideoCapture(0)
13
14 # Setup pyttsx3 and queue
15 engine = pyttsx3.init()
16 speech_queue = queue.Queue()
17
18 def speech_worker():
19 while True:
20 text = speech_queue.get()
21 if text is None:
22 break
23 engine.say(text)
24 engine.runAndWait()
25 speech_queue.task_done()
26
27 speech_thread = threading.Thread(target=speech_worker, daemon=True)
28 speech_thread.start()
29
30 def speak(text):
31 if not speech_queue.full():
32 speech_queue.put(text)
33
34 # Constants for distance calculation
35 FOCAL_LENGTH = 360
36 KNOWN_WIDTH = 60 # width of person in cm
37 SPEECH_INTERVAL = 1 # seconds between speech updates
Ln 99, Col 23 Spaces: 4 UTF-8 CRLF Python 3.13.1 Go Live Go Live


File Edit Selection View Go Run Terminal Help
object_1
EXPLORER
real time.py
OPEN EDITORS
real time.py object_
OBJECT_1
object_
.idea
.venv
33.mp4
34.mp4
bus.jpg
bus2.jpg
bus3.jpg
coco.txt
conv.py
object detection.py
output_detected.jpg
real time.py
test4.py
video with distance.py
yolov8m.pt
yolov8n.pt
OUTLINE
TIMELINE
object_ > real time.py > ...
30 def speak(text):
31 if not speech_queue.full():
32 speech_queue.put(text)
33
34 # Constants for distance calculation
35 FOCAL_LENGTH = 360
36 KNOWN_WIDTH = 60 # width of person in cm
37 SPEECH_INTERVAL = 1 # seconds between speech updates
38 last_speech_time = 0
39
40 # Load COCO class labels
41 with open("coco.txt", "r") as f:
42 class_list = f.read().splitlines()
43
44 # Main loop
45 while True:
46 ret, frame = cap.read()
47 if not ret:
48 break
49
50 # Run YOLO detection on every frame
51 results = model(frame, stream=True)
52 detection_made = False
53
54 for r in results:
55 boxes = r.boxes
56 for box in boxes:
57 cls_id = int(box.cls[0])
58 class_name = class_list[cls_id]
59
60 x1, y1, x2, y2 = map(int, box.xyxy[0])
61 width = x2 - x1
62 distance = (KNOWN_WIDTH * FOCAL_LENGTH) / width
63
64 # Safety logic
65 if distance < 50:
Ln 99, Col 23 Spaces: 4 UTF-8 CRLF Python 3.13.1 Go Live Go Live


YOLOv8 Real-Time Detection
tvmonitor
chair 42.35 cm (Unsafe)


YOLOv8 Real-Time Detection
person 66.06 cm (Safe)
person 76.33 cm (Safe)
backpack 92.70 cm
tvmonitor
cm (Safe)
342.86 cm (Safe)

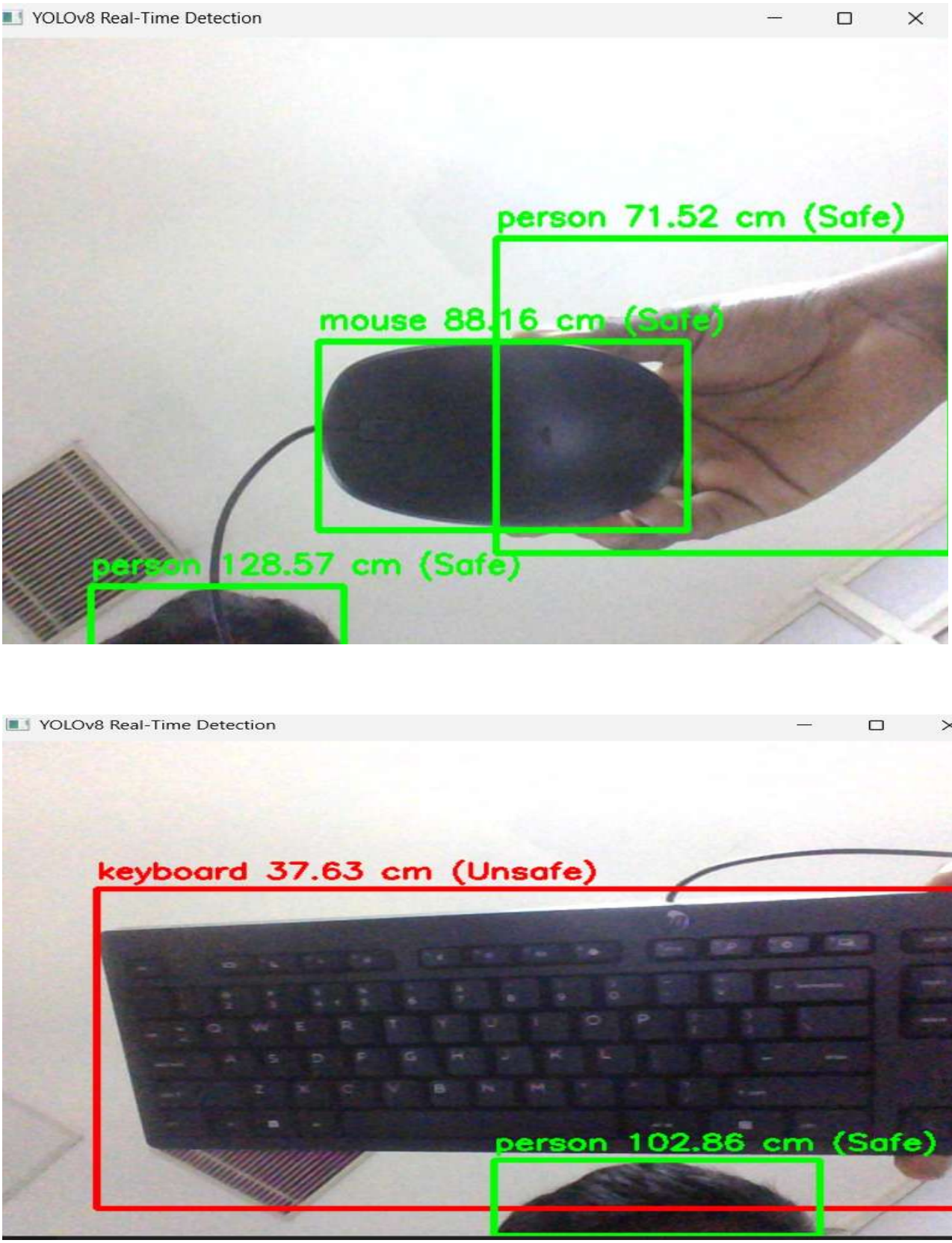
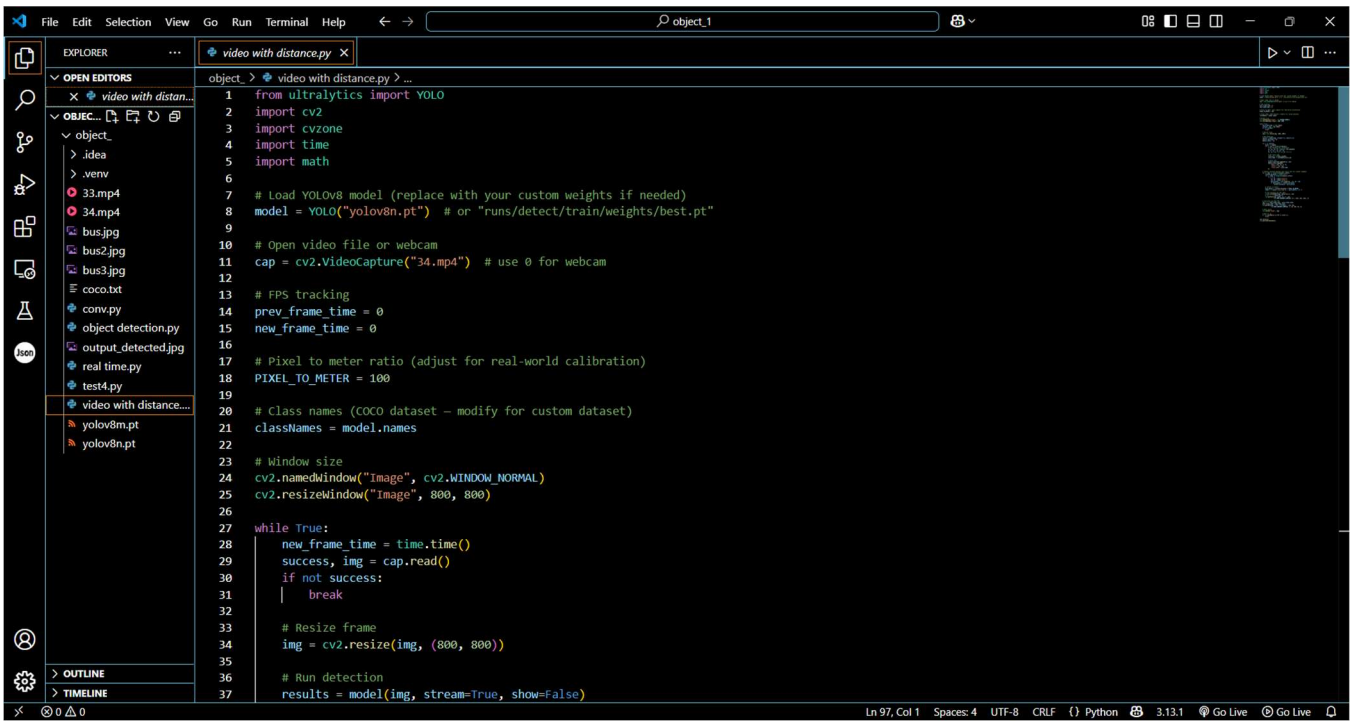


Figure 6.5 Real Time Object Detection with Distance mapping


File Edit Selection View Go Run Terminal Help
object_1
EXPLORER
video with distance.py
OPEN EDITORS
video with distan...
OBJEC...
object_
.idea
.venv
33.mp4
34.mp4
bus.jpg
bus2.jpg
bus3.jpg
coco.txt
conv.py
object detection.py
output_detected.jpg
real time.py
test4.py
video with distance....
yolov8m.pt
yolov8n.pt
OUTLINE
TIMELINE
object_ > video with distance.py > ...
from ultralytics import YOLO
import cv2
import cvzone
import time
import math
# Load YOLOv8 model (replace with your custom weights if needed)
model = YOLO("yolov8n.pt") # or "runs/detect/train/weights/best.pt"
# Open video file or webcam
cap = cv2.VideoCapture("34.mp4") # use 0 for webcam
# FPS tracking
prev_frame_time = 0
new_frame_time = 0
# Pixel to meter ratio (adjust for real-world calibration)
PIXEL_TO_METER = 100
# Class names (COCO dataset — modify for custom dataset)
classNames = model.names
# Window size
cv2.namedWindow("Image", cv2.WINDOW_NORMAL)
cv2.resizeWindow("Image", 800, 800)
while True:
new_frame_time = time.time()
success, img = cap.read()
if not success:
break
# Resize frame
img = cv2.resize(img, (800, 800))
# Run detection
results = model(img, stream=True, show=False)
Ln 97, Col 1 Spaces: 4 UTF-8 CRLF Python 3.13.1 Go Live Go Live

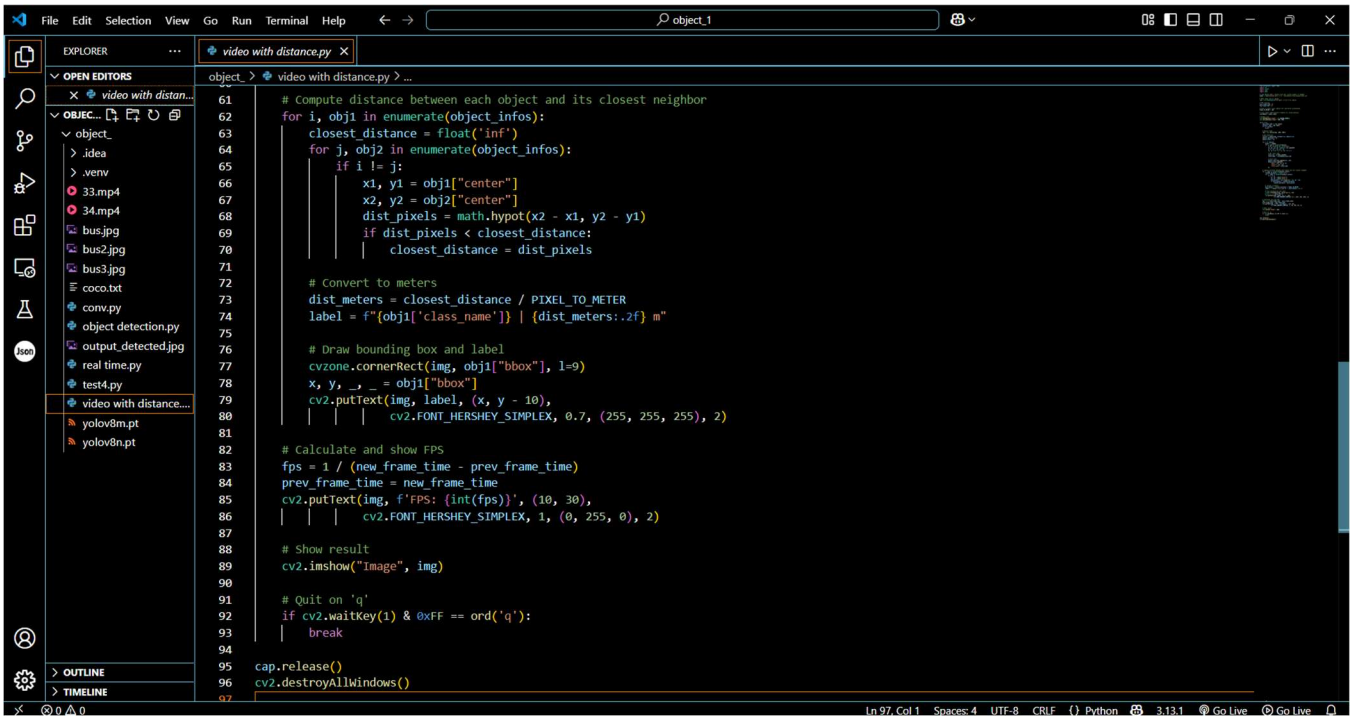
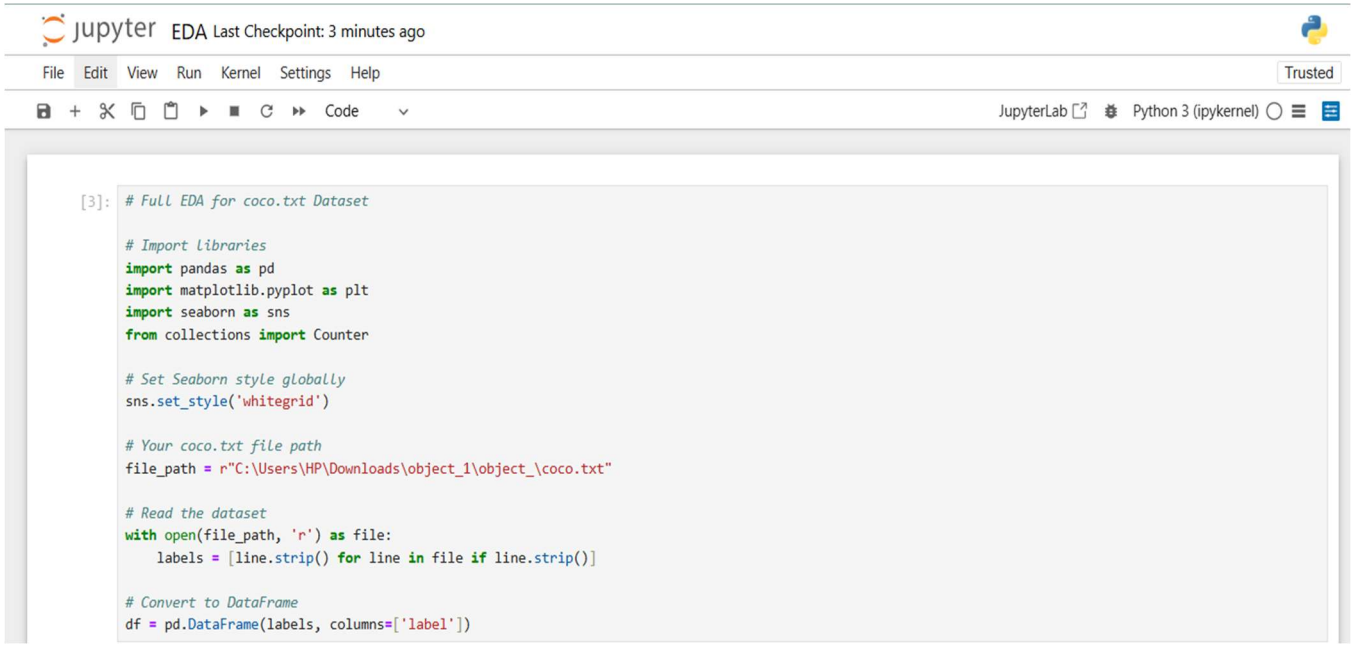

File Edit Selection View Go Run Terminal Help
object_1
EXPLORER
video with distance.py
OPEN EDITORS
video with distan...
OBJEC...
object_
.idea
.venv
33.mp4
34.mp4
bus.jpg
bus2.jpg
bus3.jpg
coco.txt
conv.py
object detection.py
output_detected.jpg
real time.py
test4.py
video with distance....
yolov8m.pt
yolov8n.pt
OUTLINE
TIMELINE
object_ > video with distance.py > ...
# Compute distance between each object and its closest neighbor
for i, obj1 in enumerate(object_infos):
closest_distance = float('inf')
for j, obj2 in enumerate(object_infos):
if i != j:
x1, y1 = obj1["center"]
x2, y2 = obj2["center"]
dist_pixels = math.hypot(x2 - x1, y2 - y1)
if dist_pixels < closest_distance:
closest_distance = dist_pixels
# Convert to meters
dist_meters = closest_distance / PIXEL_TO_METER
label = f"{obj1['class_name']} | {dist_meters:.2f} m"
# Draw bounding box and label
cvzone.cornerRect(img, obj1["bbox"], l=9)
x, y, _, _ = obj1["bbox"]
cv2.putText(img, label, (x, y - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
# Calculate and show FPS
fps = 1 / (new_frame_time - prev_frame_time)
prev_frame_time = new_frame_time
cv2.putText(img, f'FPS: {int(fps)}', (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
# Show result
cv2.imshow("Image", img)
# Quit on 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
break
cap.release()
cv2.destroyAllWindows()
Ln 97, Col 1 Spaces: 4 UTF-8 CRLF Python 3.13.1 Go Live Go Live

Figure 6.6 Object Detection using Video and Distance Mapping



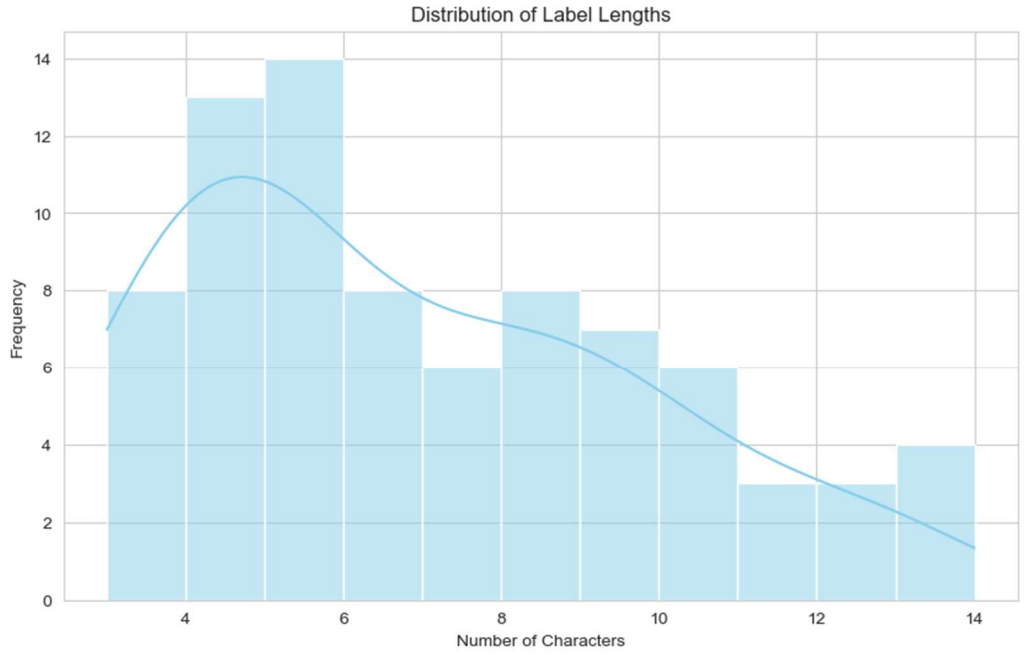

```
]:
# ============== Basic Overview ==============
print("="*40)
print("Basic Dataset Overview")
print("="*40)
print(f"Total labels: {df.shape[0]}")
print("\nFirst 10 labels:")
print(df.head(10))

# Duplicates and Missing Values
print("\nDuplicate labels count:", df.duplicated().sum())
print("Missing labels:", df.isnull().sum().values[0])
```

```
========================================
Basic Dataset Overview
========================================
Total labels: 80

First 10 labels:
          label  word_count  char_count
0        person           1           6
1       bicycle           1           7
2           car           1           3
3     motorbike           1           9
4     aeroplane           1           9
5           bus           1           3
6         train           1           5
7         truck           1           5
8          boat           1           4
9  traffic light           2          13

Duplicate labels count: 0
Missing labels: 0
```

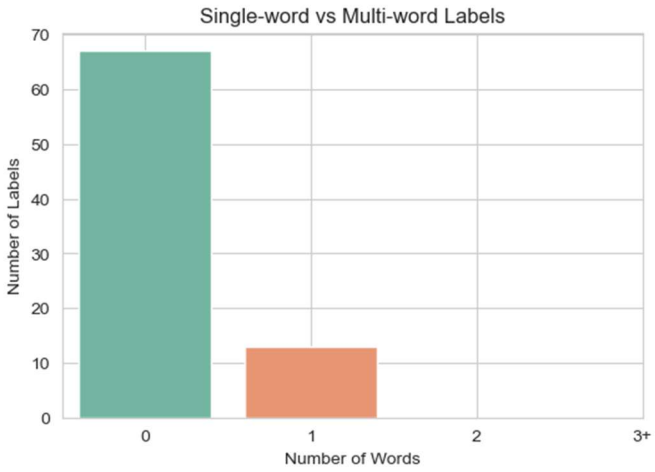
[5]:
```python
# ============== Word and Character Analysis ==============
df['word_count'] = df['label'].apply(lambda x: len(x.split()))
df['char_count'] = df['label'].apply(len)

single_word_labels = df[df['word_count'] == 1]
multi_word_labels = df[df['word_count'] > 1]

print("\nSingle-word labels:", single_word_labels.shape[0])
print("Multi-word labels:", multi_word_labels.shape[0])

print("\nShortest label:", df.loc[df['char_count'].idxmin()])
print("Longest label:", df.loc[df['char_count'].idxmax()])

# Most common words inside labels
all_words = ' '.join(df['label']).split()
word_counter = Counter(all_words)

print("\nMost common words inside labels:")
for word, freq in word_counter.most_common(10):
    print(f"{word}: {freq}")
```

```
Single-word labels: 67
Multi-word labels: 13

Shortest label: label         car
word_count      1
char_count      3
Name: 2, dtype: object
Longest label: label         baseball glove
word_count                2
char_count               14
Name: 35, dtype: object

Most common words inside labels:
dog: 2
bear: 2
baseball: 2
person: 1
bicycle: 1
car: 1
motorbike: 1
aeroplane: 1
bus: 1
train: 1
```

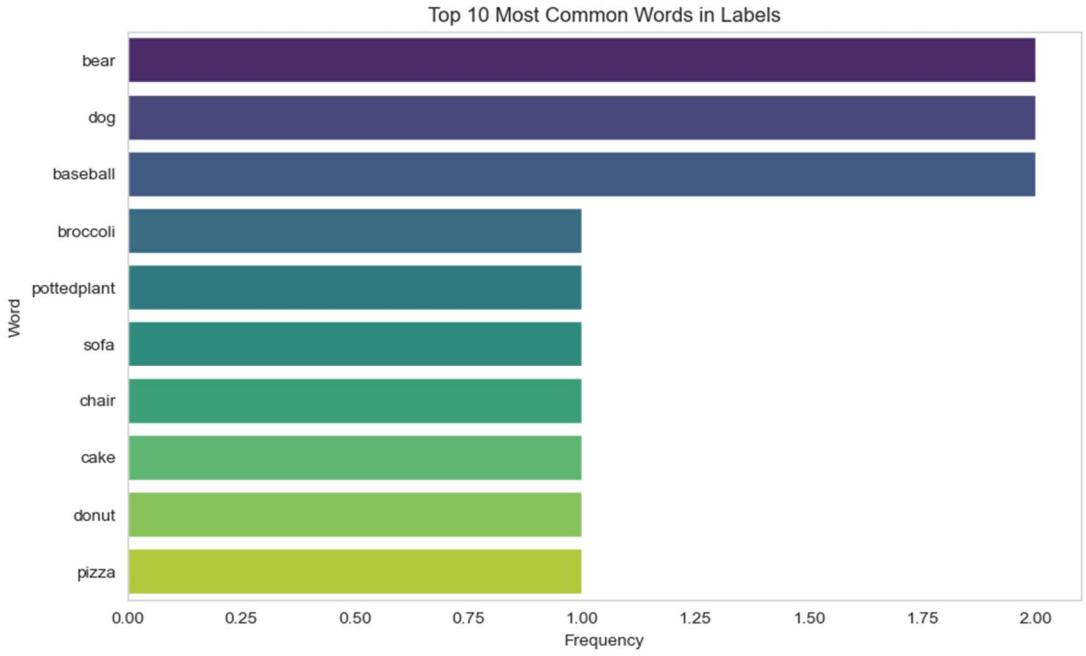
```python
# 1. Label Length distribution
plt.figure(figsize=(10,6))
sns.histplot(df['char_count'], bins=range(df['char_count'].min(), df['char_count'].max() + 1), kde=True, color="skyblue")
plt.title('Distribution of Label Lengths')
plt.xlabel('Number of Characters')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```




```python
# 2. Word count (Single-word vs Multi-word)
plt.figure(figsize=(6,4))
sns.countplot(x='word_count', hue='word_count', data=df, palette='Set2', legend=False)
plt.title('Single-word vs Multi-word Labels')
plt.xlabel('Number of Words')
plt.ylabel('Number of Labels')
plt.xticks([0,1,2,3], ['0','1','2','3+'])
plt.grid(True)
plt.show()
```



```python
# 3. Top 10 most common words in labels
common_words_df = pd.DataFrame(word_counter.items(), columns=['word', 'count']).sort_values(by='count', ascending=False)
plt.figure(figsize=(10,6))
sns.barplot(x='count', y='word', hue='word', data=common_words_df.head(10), palette='viridis', dodge=False, legend=False)
plt.title('Top 10 Most Common Words in Labels')
plt.xlabel('Frequency')
plt.ylabel('Word')
plt.grid(axis='x')
plt.show()
```




```python
# ============== Summary Report ==============
print("\n" + "="*40)
print("Summary Report")
print("="*40)
print(f"Total labels: {df.shape[0]}")
print(f"Unique labels: {df['label'].nunique()}")
print(f"Duplicate labels: {df.duplicated().sum()}")
print(f"Single-word labels: {single_word_labels.shape[0]}")
print(f"Multi-word labels: {multi_word_labels.shape[0]}")
print(f"Average characters per label: {df['char_count'].mean():.2f}")
print(f"Most common word inside labels: '{common_words_df.iloc[0]['word']}' used {common_words_df.iloc[0]['count']} times")
```

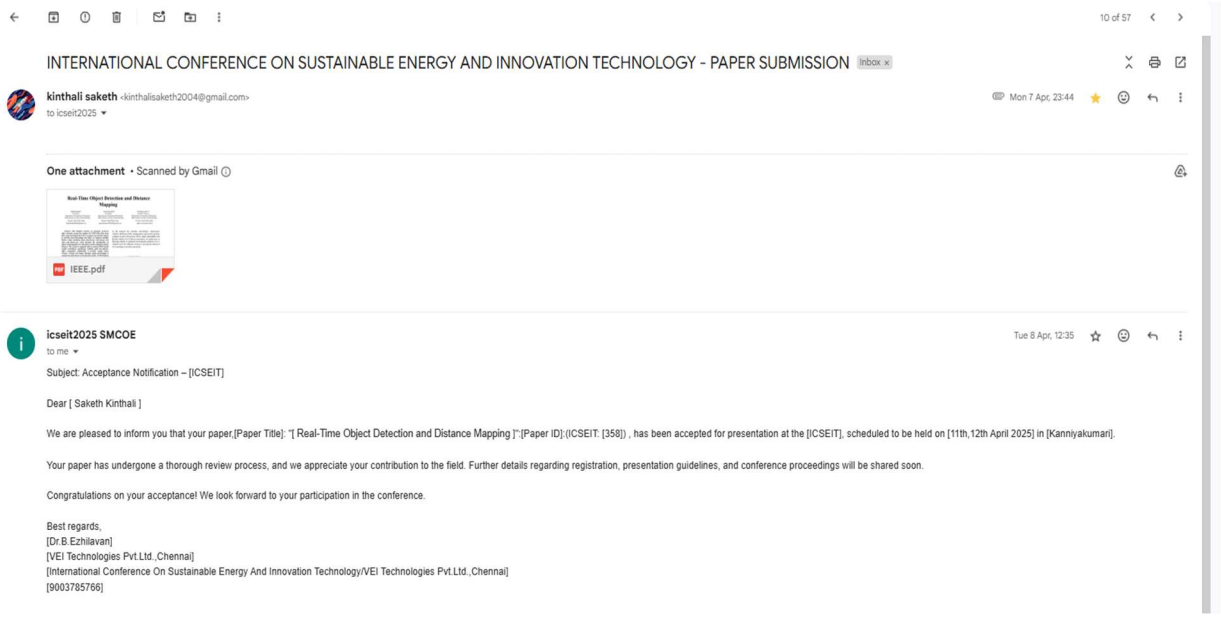
```
========================================
Summary Report
========================================
Total labels: 80
Unique labels: 80
Duplicate labels: 0
Single-word labels: 67
Multi-word labels: 13
Average characters per label: 6.81
Most common word inside labels: 'bear' used 2 times
```

Figure 6.7 Exploratory Data Analysis

APPENDIX B

CONFERENCE PUBLICATION

Our paper on Real-Time Object Detection and Distance Mapping was submitted to the International Conference on Emerging Solutions in Information Technology (ICESIT) and the journal JETNR, where it has been accepted. Additionally, it has been submitted to ICICNIS and is currently under review.



10 of 57

INTERNATIONAL CONFERENCE ON SUSTAINABLE ENERGY AND INNOVATION TECHNOLOGY - PAPER SUBMISSION Inbox ×

kinthali saketh <kinthalisaketh2004@gmail.com>
to icseit2025

Mon 7 Apr, 23:44

One attachment • Scanned by Gmail

IEEE.pdf

icseit2025 SMCOE
to me

Tue 8 Apr, 12:35

Subject: Acceptance Notification – [ICSEIT]

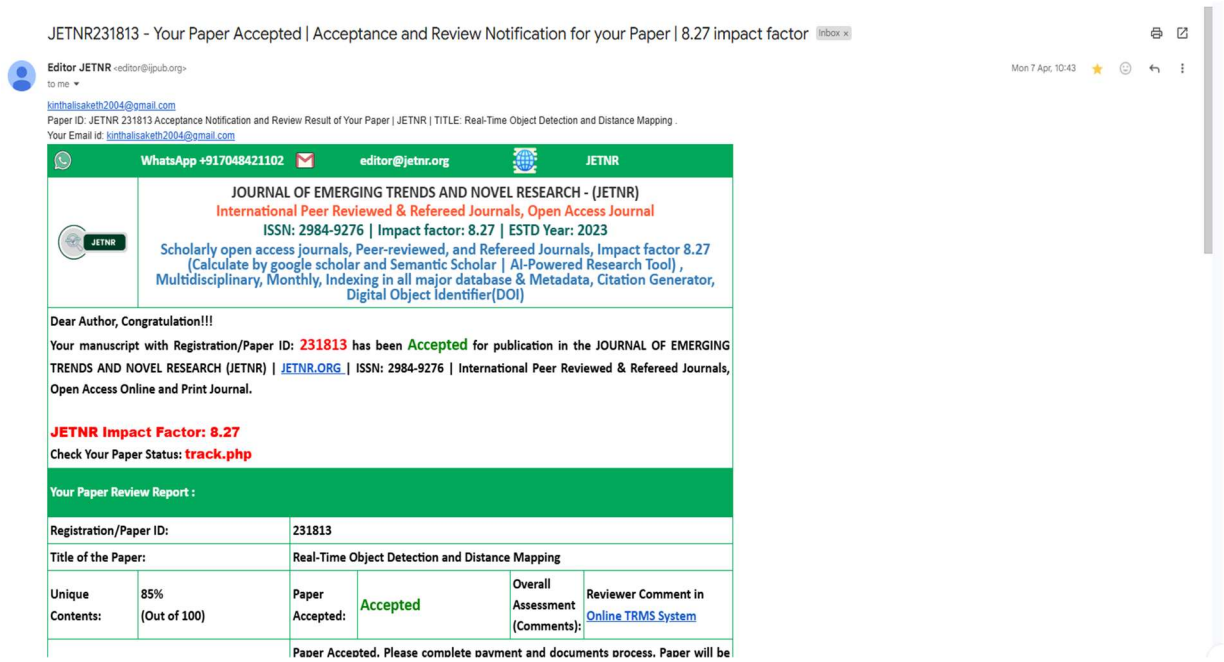
Dear [Saketh Kinthali]

We are pleased to inform you that your paper,[Paper Title]: "[Real-Time Object Detection and Distance Mapping]":(Paper ID):(ICSEIT: [358]) , has been accepted for presentation at the [ICSEIT], scheduled to be held on [11th,12th April 2025] in [Kanniyakumari].

Your paper has undergone a thorough review process, and we appreciate your contribution to the field. Further details regarding registration, presentation guidelines, and conference proceedings will be shared soon.

Congratulations on your acceptance! We look forward to your participation in the conference.

Best regards,
[Dr.B.Ezhilavan]
[VEI Technologies Pvt.Ltd.,Chennai]
[International Conference On Sustainable Energy And Innovation Technology/VEI Technologies Pvt.Ltd.,Chennai]
[9003785766]



JETNR231813 - Your Paper Accepted | Acceptance and Review Notification for your Paper | 8.27 impact factor Inbox ×

Editor JETNR <editor@ijpub.org> Mon 7 Apr, 10:43

to me

kinthalisaketh2004@gmail.com

Paper ID: JETNR 231813 Acceptance Notification and Review Result of Your Paper | JETNR | TITLE: Real-Time Object Detection and Distance Mapping .

Your Email id: kinthalisaketh2004@gmail.com

WhatsApp +917048421102 editor@jetnr.org JETNR

JOURNAL OF EMERGING TRENDS AND NOVEL RESEARCH - (JETNR)

International Peer Reviewed & Refereed Journals, Open Access Journal

ISSN: 2984-9276 | Impact factor: 8.27 | ESTD Year: 2023

Scholarly open access journals, Peer-reviewed, and Refereed Journals, Impact factor 8.27 (Calculate by google scholar and Semantic Scholar | AI-Powered Research Tool) , Multidisciplinary, Monthly, Indexing in all major database & Metadata, Citation Generator, Digital Object Identifier(DOI)

Dear Author, Congratulation!!!

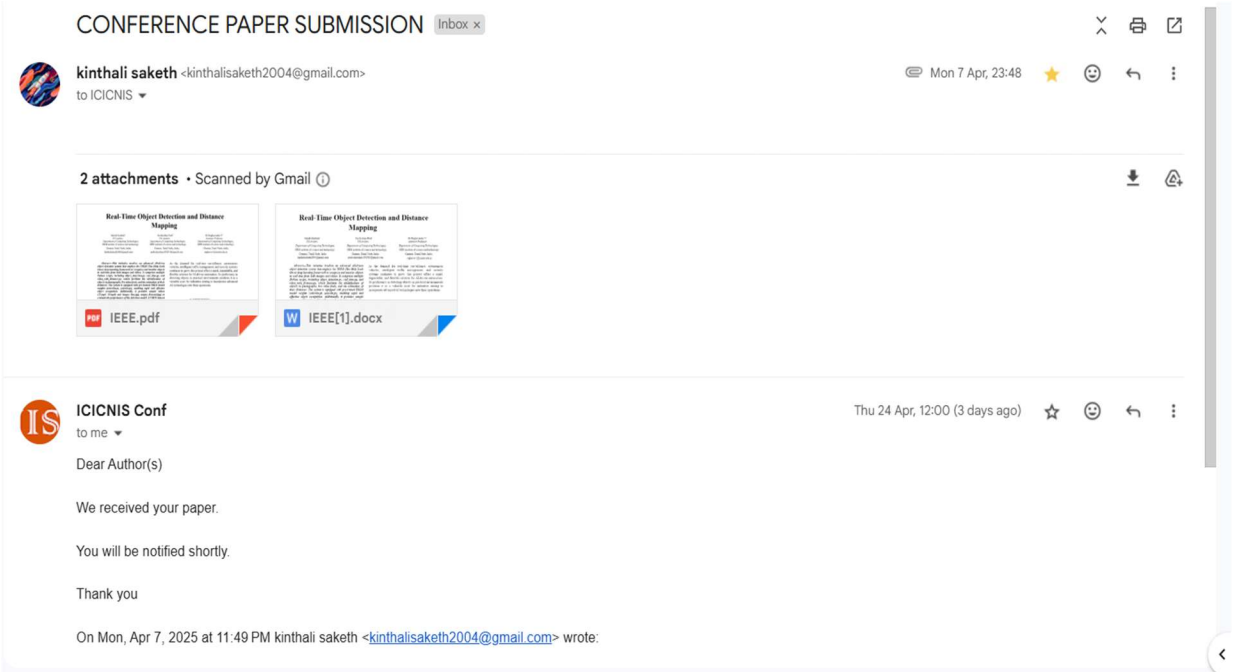
Your manuscript with Registration/Paper ID: 231813 has been Accepted for publication in the JOURNAL OF EMERGING TRENDS AND NOVEL RESEARCH (JETNR) | JETNR.ORG | ISSN: 2984-9276 | International Peer Reviewed & Refereed Journals, Open Access Online and Print Journal.

JETNR Impact Factor: 8.27

Check Your Paper Status: track.php

Your Paper Review Report :

Registration/Paper ID:		231813			
Title of the Paper:		Real-Time Object Detection and Distance Mapping			
Unique Contents:	85% (Out of 100)	Paper Accepted:	Accepted	Overall Assessment (Comments):	Reviewer Comment in Online TRMS System
		Paper Accepted. Please complete payment and documents process. Paper will be			



CONFERENCE PAPER SUBMISSION Inbox ×

kinthali saketh <kinthalisaketh2004@gmail.com> Mon 7 Apr, 23:48

to ICICNIS

2 attachments • Scanned by Gmail

Real-Time Object Detection and Distance Mapping

IEEE.pdf

Real-Time Object Detection and Distance Mapping

IEEE[1].docx

ICICNIS Conf Thu 24 Apr, 12:00 (3 days ago)

to me

Dear Author(s)

We received your paper.

You will be notified shortly.

Thank you

On Mon, Apr 7, 2025 at 11:49 PM kinthali saketh <kinthalisaketh2004@gmail.com> wrote:

Figure 7. Conference Paper Submission

Real-Time Object Detection and Distance Mapping

Dr Raghavendra V [1]
Assistant Professor
Department of Computing Technologies,
SRM institute of science and technology,
Chennai, Tamil Nadu, India,
raghavev1@srmist.edu.in

Saketh Kinthali[2]
UG student,
Department of Computing Technologies,
SRM institute of science and technology,
Chennai, Tamil Nadu, India,
kinthalisaketh2004@gmail.com

Sai Krishna Pudi[3]
UG Student,
Department of Computing Technologies,
SRM institute of science and technology,
Chennai, Tamil Nadu, India,
pudisaikrishna1592918@gmail.com

Abstract*—This initiative involves an advanced AI-driven object detection system that employs the YOLO (You Only Look Once) deep learning framework to recognize and monitor objects in real-time from both images and videos. It comprises multiple Python scripts, including object_detection.py, real_time.py, and video_with_distance.py, which facilitate the identification of objects in photographs, live video feeds, and the estimation of their distances. The system is equipped with pre-trained YOLO model weights (yolov8m.pt, yolov8n.pt), enabling rapid and effective object recognition. Additionally, it provides sample videos (33.mp4, 34.mp4) and images (bus.jpg, output_detected.jpg) to evaluate the performance of the detection model. A COCO dataset file (coco.txt) is included, signifying that the model has been trained to identify a diverse range of common objects. Furthermore, other Python scripts (conv.py, test4.py) appear to be utilized for data conversion, testing, or enhancing the system's capabilities. This project holds significant potential for applications in real-time surveillance, autonomous vehicles, intelligent traffic management, security systems, and AI-driven automation, thereby improving the efficiency of object detection and tracking across various practical scenarios.

Key Words*— Object Detection, YOLO (You Only Look Once), Deep Learning, Computer Vision, Real-Time Detection, AI-powered Automation, COCO Dataset, Bounding Box Annotation, Image Recognition, Video Processing, Machine Learning, Autonomous Systems, Smart Surveillance, Security Monitoring, Artificial Intelligence

I. INTRODUCTION

Artificial intelligence and deep learning are revolutionizing our interactions with technology, enhancing the intelligence and capabilities of machines. A notable advancement in this domain is object detection, which enables computers to identify and monitor objects within images and videos. This project features an AI-driven object detection system that employs the YOLO (You Only Look Once) deep learning model to recognize and analyze objects in real time. By utilizing pre-trained YOLO model weights (yolov8m.pt, yolov8n.pt), the system can swiftly and accurately detect a wide range of objects. It includes several Python scripts designed to process images, live video feeds, and even estimate the distance to detected objects. To evaluate its effectiveness, the project provides sample videos (33.mp4, 34.mp4) and images (bus.jpg, output_detected.jpg), demonstrating the model's performance across various scenarios. Furthermore, it utilizes the COCO dataset (coco.txt), which enhances the model's capability to identify common objects.

As the demand for real-time surveillance, autonomous vehicles, intelligent traffic management, and security systems continues to grow, this project offers a rapid, dependable, and flexible solution for AI-driven automation. Its proficiency in detecting objects in practical environments positions it as a valuable asset for industries aiming to incorporate advanced AI technologies into their operations.

II. MOTIVATION

As technology progresses, the significance of AI-driven automation and computer vision in daily life is becoming increasingly prominent. Applications such as security surveillance, autonomous vehicles, and intelligent traffic management systems require machines to identify and monitor objects in real time to facilitate swift and accurate decision-making. This project was motivated by the demand for a rapid, dependable, and efficient object detection system capable of functioning seamlessly in various environments.

Many conventional object detection techniques face challenges related to speed and precision, rendering them less suitable for practical applications. This is where the YOLO (You Only Look Once) deep learning model proves advantageous, as it is engineered for quick and accurate object detection, making it ideal for real-time scenarios.

Our objective with this project is to leverage the capabilities of YOLO to create a system that can identify and track objects in both images and live video streams, while also estimating distances.

By utilizing pre-trained YOLO models, COCO dataset labels, and sample test files, this initiative offers a practical and accessible solution for those interested in incorporating AI into their projects. Whether aimed at enhancing security, optimizing traffic monitoring, or advancing smart automation, this system is designed to improve the effectiveness and reliability of AI-driven technologies in real-world settings.

III. DATA SET

A large collection of photos commonly used in computer vision applications such as object detection, image segmentation, and caption creation is called the COCO

Figure 8. Cover Page of Conference Paper

APPENDIX C

PLAGIARISM REPORT

Raghavendra V

IEEE[1]_No _ref.docx

Report_1

CTech

SRM Institute of Science & Technology

Document Details

Submission ID

trn:oid:::1:3205716778

Submission Date

Apr 5, 2025, 1:02 PM GMT+5:30

Download Date

Apr 5, 2025, 1:04 PM GMT+5:30

File Name

IEEE_1__No__ref.docx

File Size

1.1 MB

5 Pages

2,900 Words

18,396 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

9 Not Cited or Quoted 3%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources

1% Publications

1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Student papers

University of San Diego <1%

2 Publication

Y. Cui. "Nanowire Nanosensors for Highly Sensitive and Selective Detection of Bio... <1%

3 Internet

docs.cvat.ai <1%

4 Publication

Jiaxu Leng, Ying Liu, Zhihui Wang, Haibo Hu, Xinbo Gao. "CrossNet: Detecting Obj... <1%

5 Internet

www.mdpi.com <1%

6 Internet

codersera.com <1%

7 Internet

discovery.researcher.life <1%